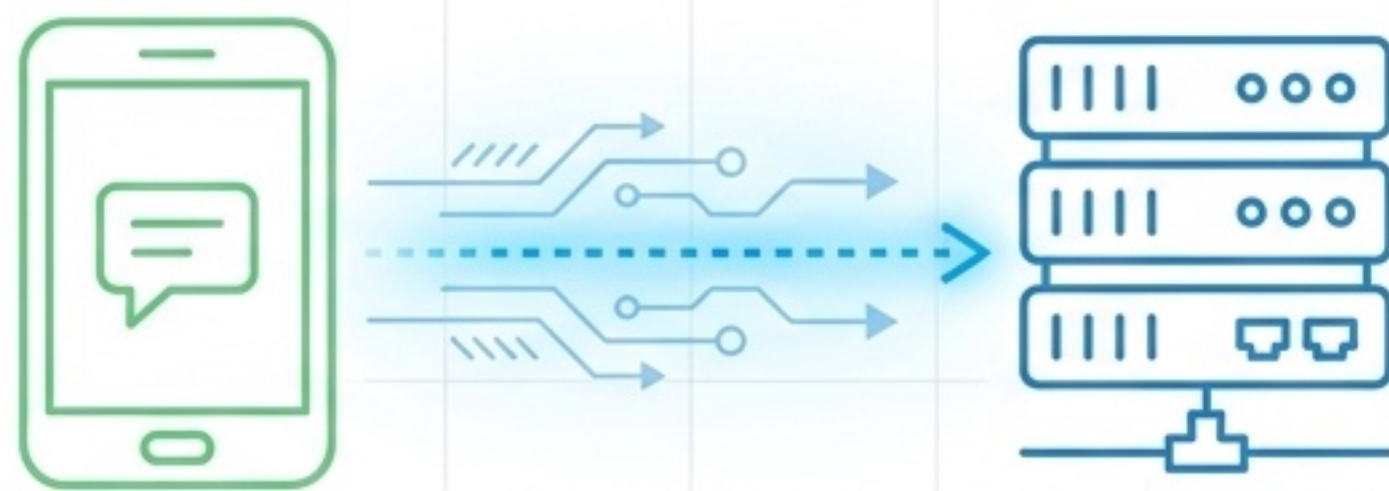




React Native 串接 FastAPI

全端開發教學-4：聊天 App 延伸補充

基礎 **CRUD** 走向實務系統架構的關鍵演進

延伸主題摘要：後端



資料模型與驗證

聊天 App 領域模型：User, Friendship, Message, ChatSummary

Pydantic Field 驗證：
min_length, max_length, Optional, date



API 與核心機制

註冊、登入、個人資料、好友、聊天訊息等非 CRUD 核心 API

UUID 產生 ID、UTC ISO 時間戳

HTTP 401、403、409 與 detail 錯誤訊息



資料結構與防護

public_user 回傳資料遮罩，避免 password 外洩 🛡️

set[tuple[str, str]] 表示雙向好友關係

list comprehension、max()、sorted()

seed endpoint 產生課堂測試資料 🌱

延伸主題摘要：前端



狀態管理 與安全

- `AuthContext`、`Provider`、`useContext`、`useMemo`、客製 `useAuth` hook
- 安全輸入與表單屬性：`secureTextEntry`、`autoCapitalize`、`multiline`
- 條件渲染、樂觀更新、錯誤恢復與 `loading disabled` 狀態



路由與 API 串接

- `Expo Router` 群組路由：`(tabs)`、`Tabs`、`Redirect`、`Stack.Screen`
- `apiRequest<T>` 泛型 API client、`RequestOptions`、`unknown body`
- `expo-constants` `hostUri` 動態推導 `API_BASE_URL`

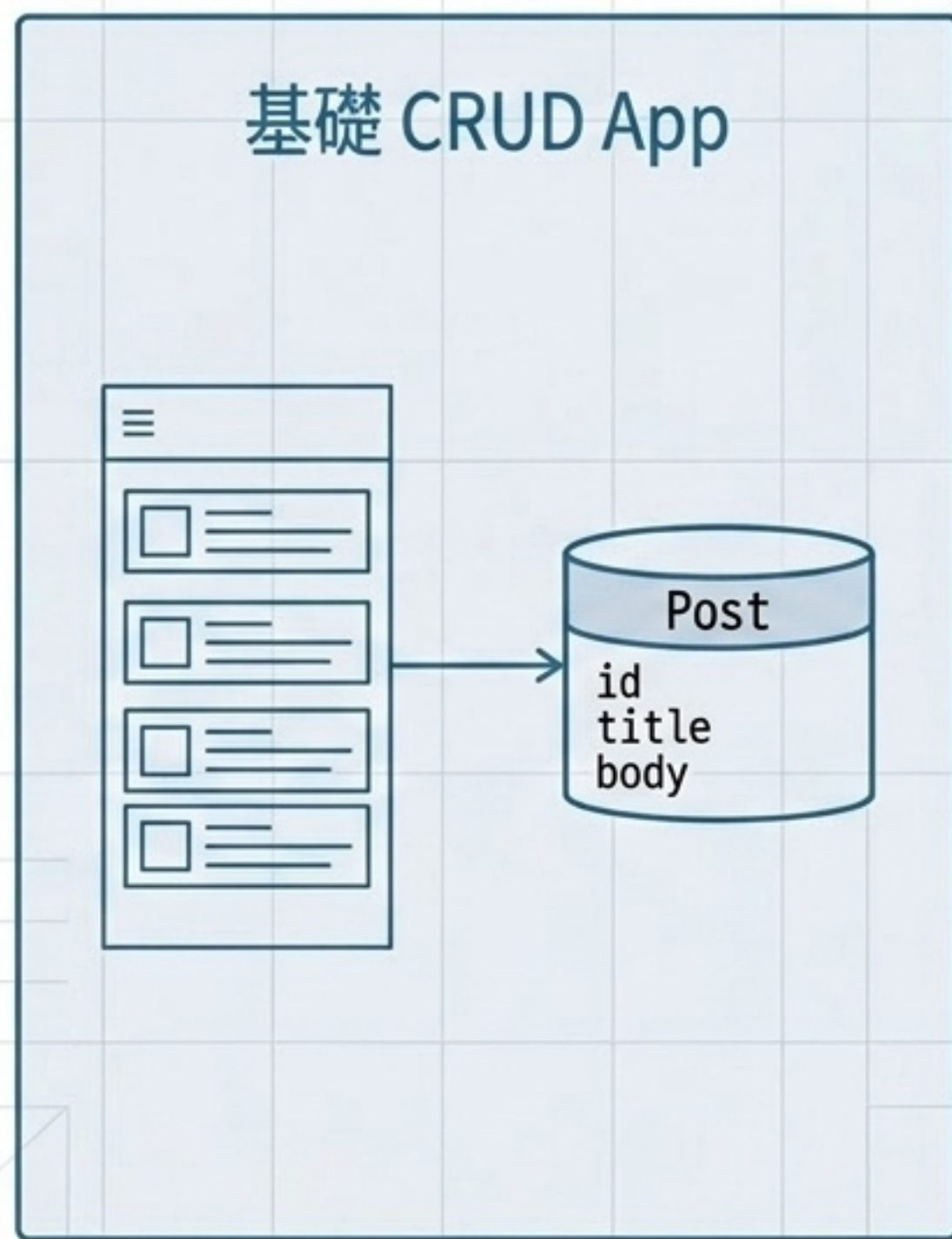


UI 元件進階

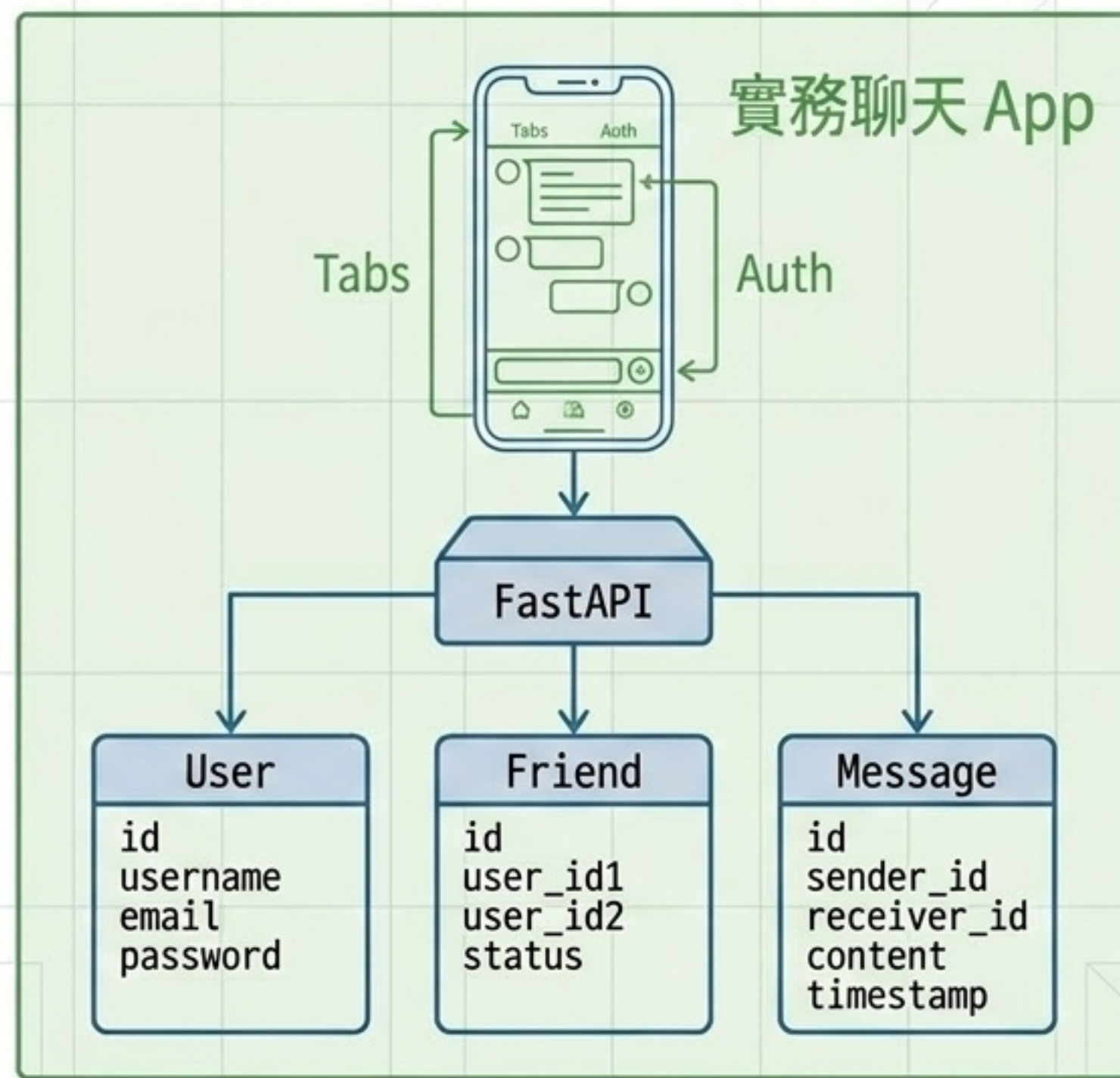
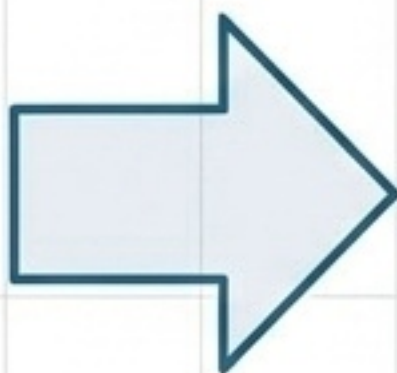
- `useFocusEffect` 與 `useCallback`：畫面重新聚焦時重新載入資料
- `FlatList` 進階屬性：`ListEmptyComponent`、`numberOfLines`
- `KeyboardAvoidingView`、`SafeAreaView`、`Image` 頭像與 `fallback UI`

從文章 CRUD 延伸到聊天 App

將「文章 CRUD 玩具」升級成「多資料模型、多頁面狀態、多 API 流程」的真實系統

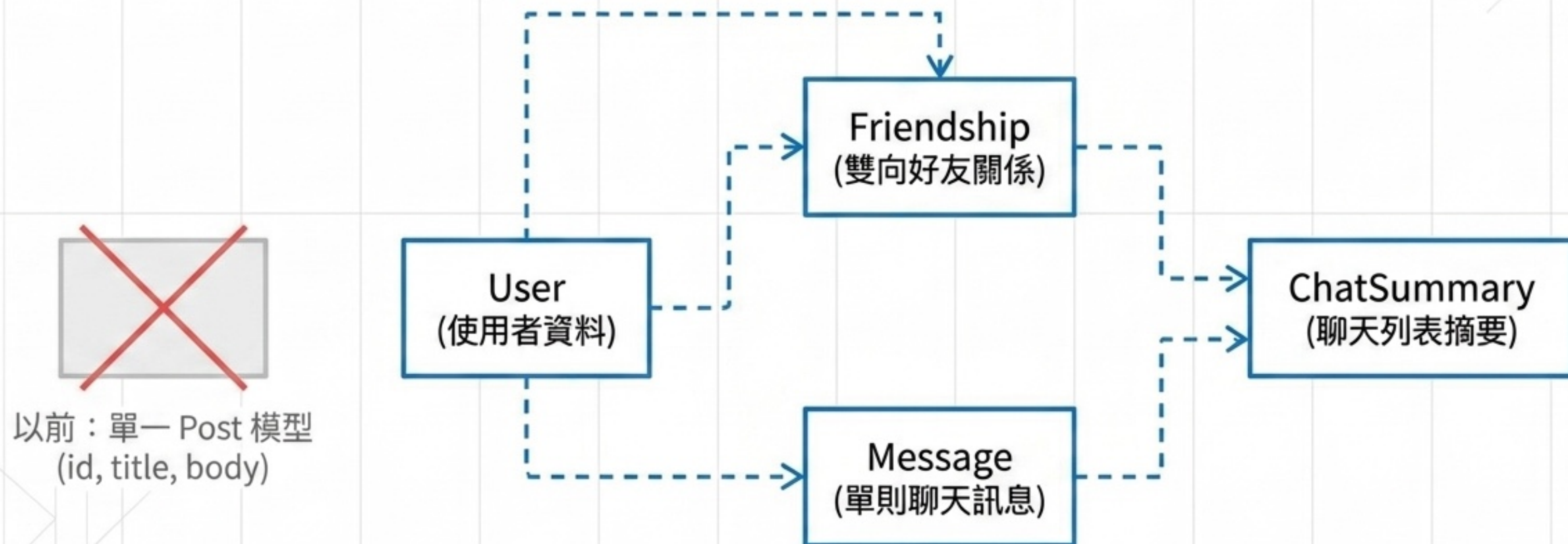


使用者註冊與登入
登入權限防護
好友關聯
聊天列表與訊息
全域狀態管理



聊天 App 需要的資料模型

當 App 功能變多，後端不只是增加 endpoint，而是要先重新思考資料庫結構的關聯性。



User 型別與公開資料

資料庫中的資料，不等於 API 應該回傳的資料。

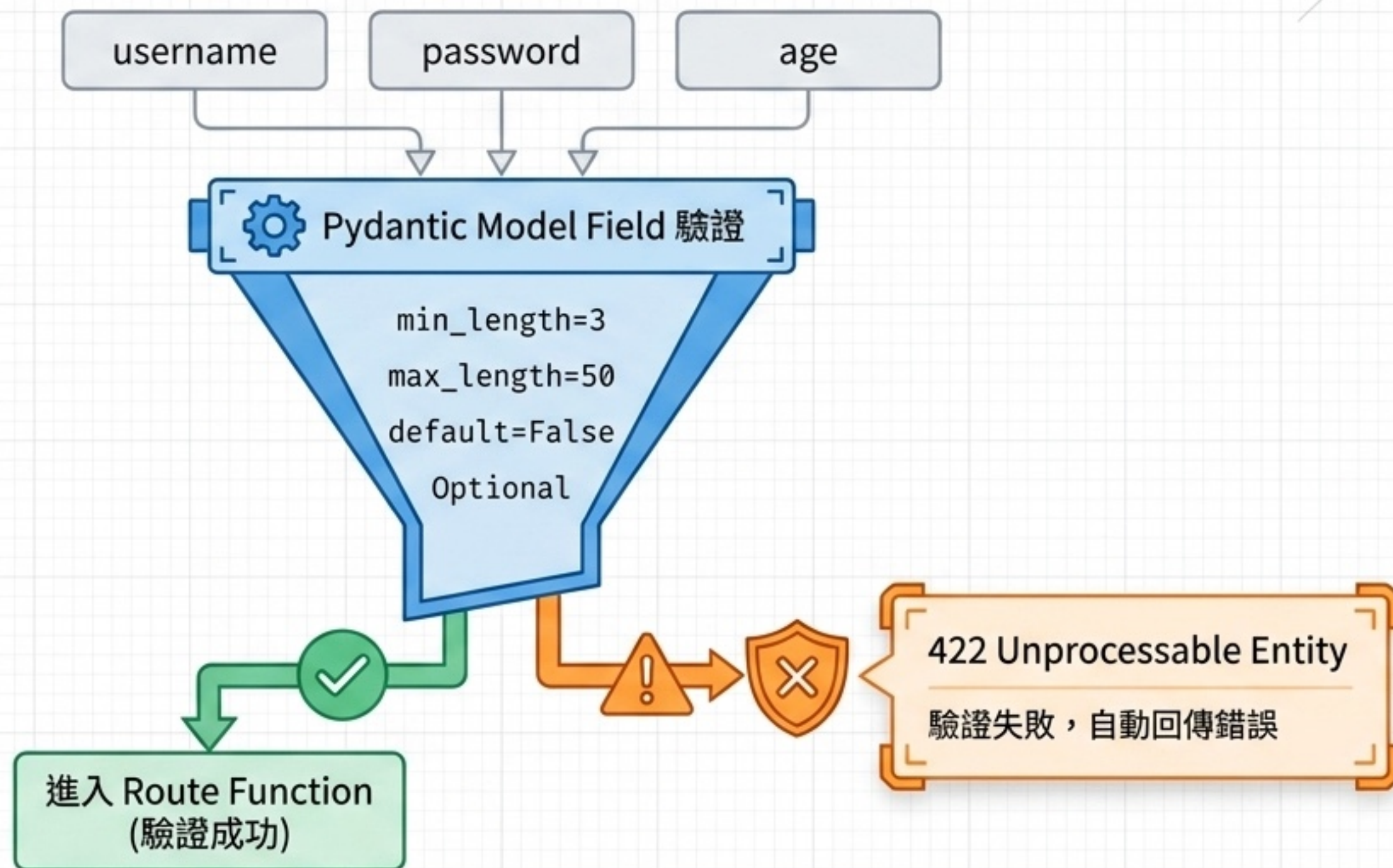
public_user() 用途：

1. 過濾機密密碼
2. 統一 API 回傳格式
3. 對齊前端 User 型別



Pydantic Field 欄位驗證

FastAPI 會自動攔截不合法的 request，前端不用自己檢查所有格式。



Optional 與 None 的意義

資料格式不是只有「有值」，必須設計「沒有值」時的前後端行為。

Python (FastAPI)



`display_name: Optional[str]`



`birthday: Optional[date]`



`avatar_url: Optional[str]`

完美對接



TypeScript (React Native)



`displayName: string | null`



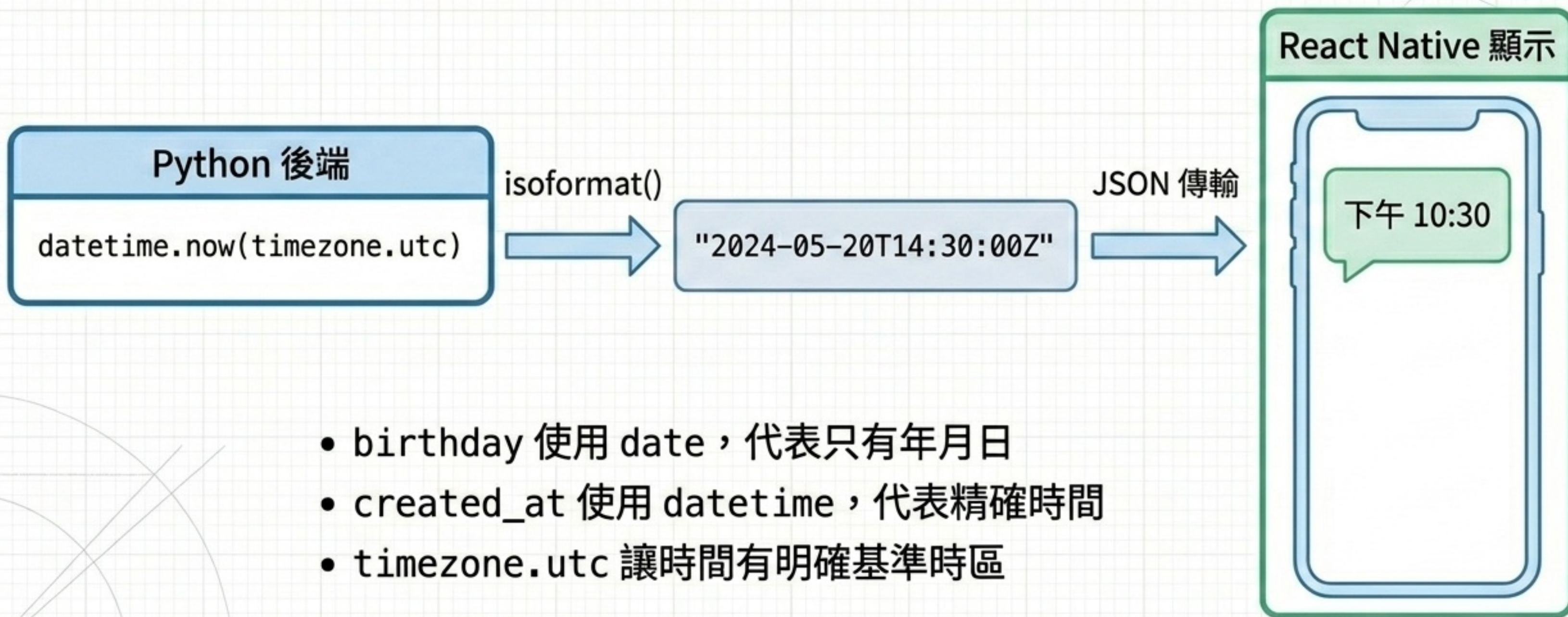
`birthday: string | null`



`avatarUrl: string | null`

日期欄位與 ISO 格式

前後端交換時間資料時，不傳送 Python 物件，而是轉換為精準的標準字串。



UUID 產生不重複 ID

擺脫 1, 2, 3 全域遞增限制，應對海量訊息與安全防護。

優點：

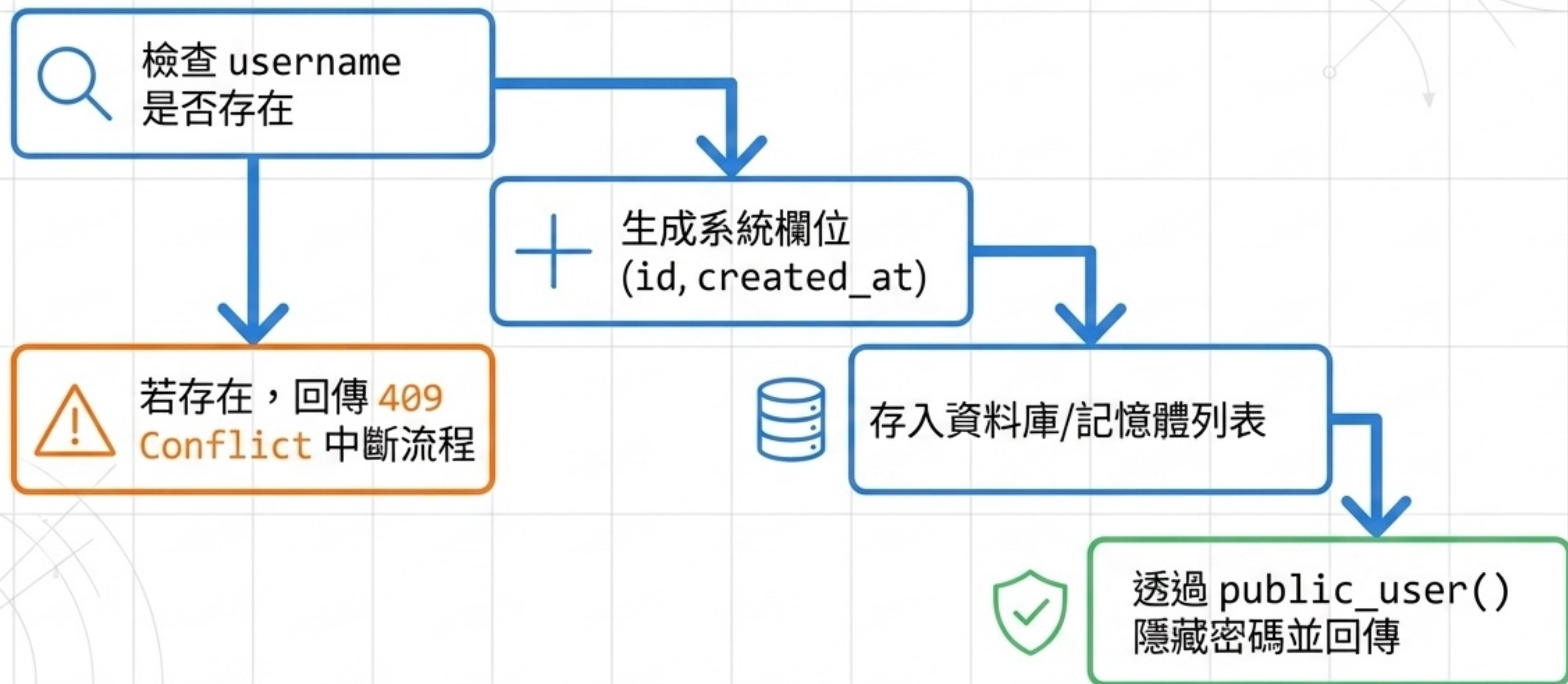
- 1. 不依賴資料庫遞增計數器
- 2. 避免與既有資料撞號
- 3. 極速產生安全識別碼



課堂示範中，ID 可截斷為前 8 碼以利閱讀；實務產品保留完整 UUID。

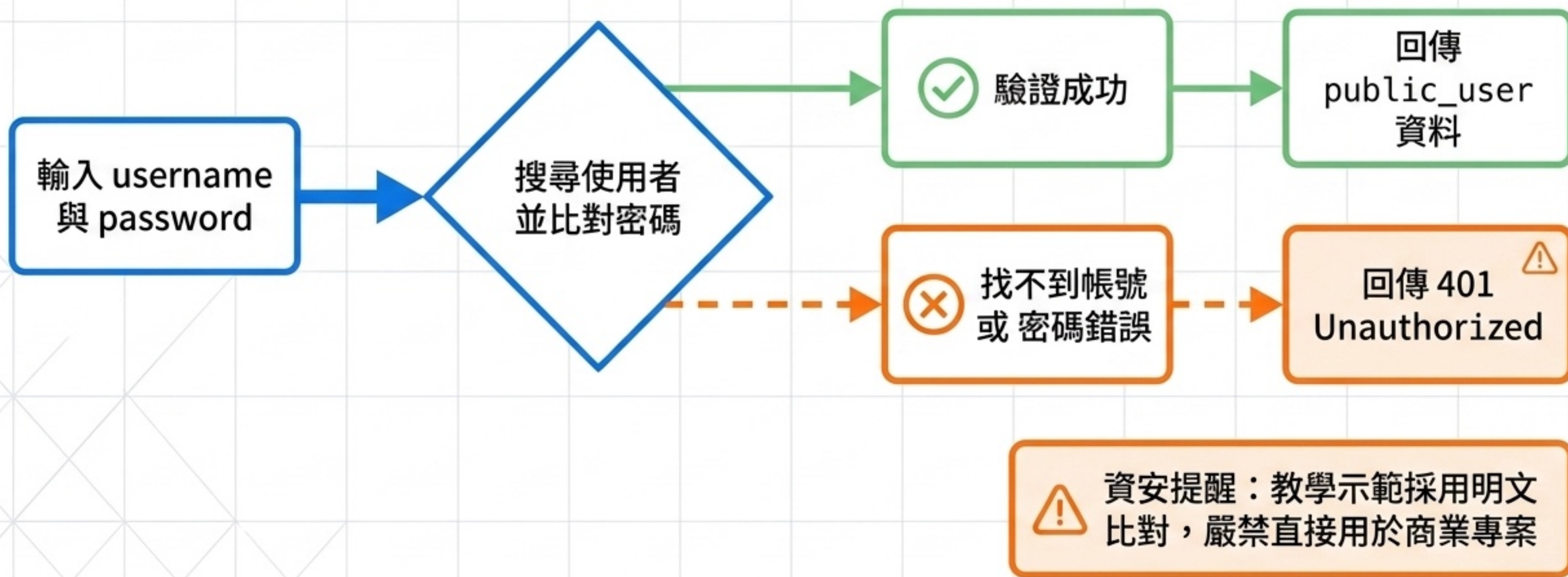
註冊 API：商業規則與建立

POST 請求不只是新增資料，更包含了驗證、攔截與資料轉換流程。



登入 API：核心驗證邏輯

課堂簡化版直接比對字串，實務上必須使用 bcrypt 雜湊搭配 Token。



HTTPException 與狀態碼設計

FastAPI 可中斷請求並直接回傳對應的語意化錯誤給前端。



使用者操作不合法
(例如加自己為好友)



帳號或密碼錯誤
(未授權)



非好友，拒絕讀取訊息
(權限不足)



非好友，拒絕讀訊息
(權限不足)



找不到指定的使用者
(資源遺失)



帳號已被註冊
(資料衝突)

Helper Function 讓 Route 更乾淨

讓 API 路由負責描述高層次流程，讓 Helper Function 處理底層細節。

🔧 好處：提高可讀性、集中管理邏輯、錯誤處理一致。

主 Route (API 流程)

1. 驗證
2. 拿資料
3. 回傳

Helper Functions (業務細節)

🔧 find_user()

🔧 find_user_by_username()

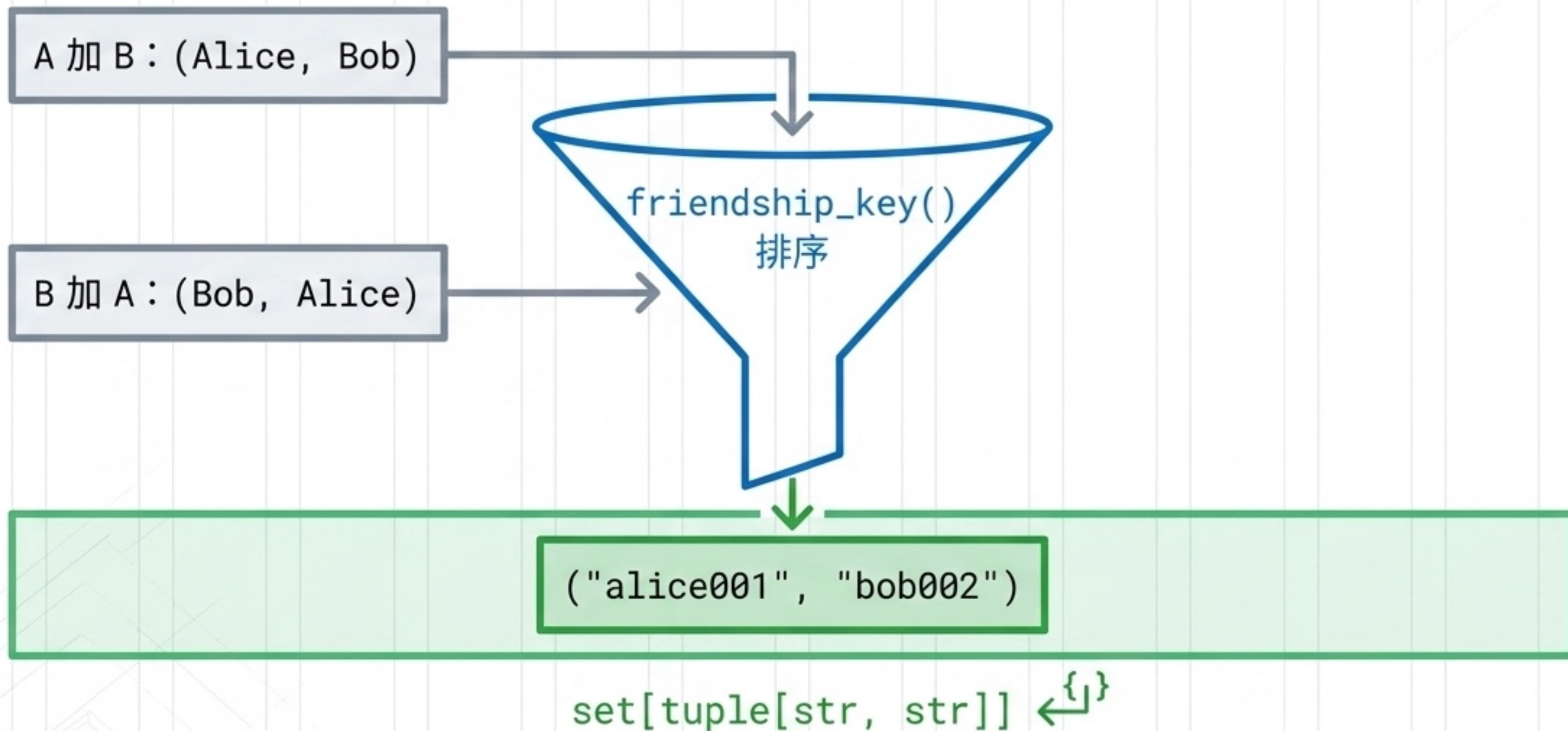
🔧 friendship_key()

🔧 ensure_friends()

🔧 message_between()

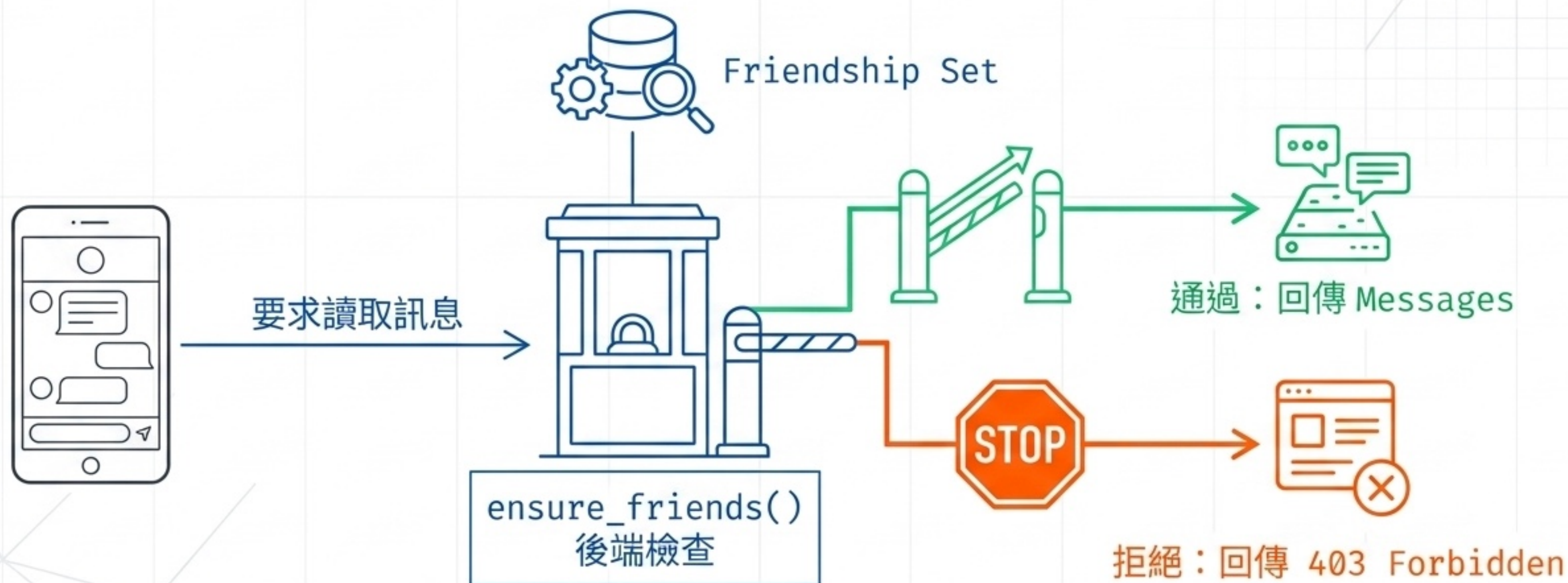
好友關係：set 與 tuple 資料結構

確保 Alice 加 Bob 與 Bob 加 Alice 不會產生重複關係。



確保只有好友能聊天（後端防護）

前端隱藏按鈕並不安全，真正的授權規則必須在後端再次攔截檢查。



List Comprehension 過濾資料

從陣列中挑選雙方訊息，一行程式完成 filter + collect。

```
[msg for msg in messages if is_between(msg, user_a, user_b)]
```

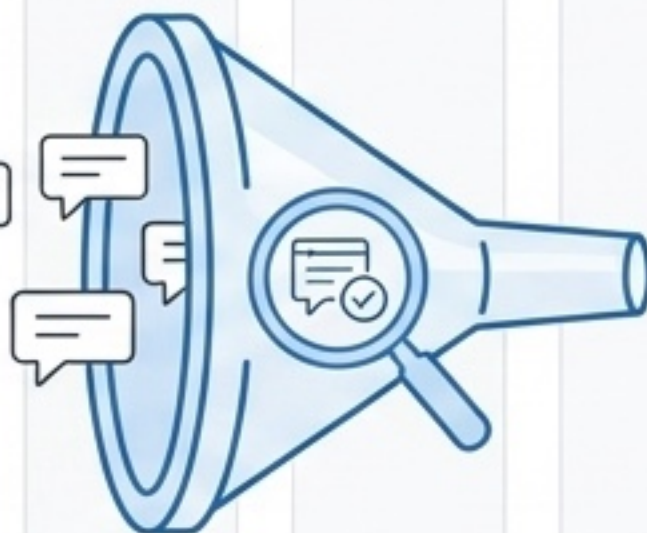
收集結果
(產生新陣列)

迭代來源
(遍歷迴圈)

條件過濾
(只留符合者)



raw messages



targeted messages
目標訊息

聊天列表：max 與 sorted 演算法

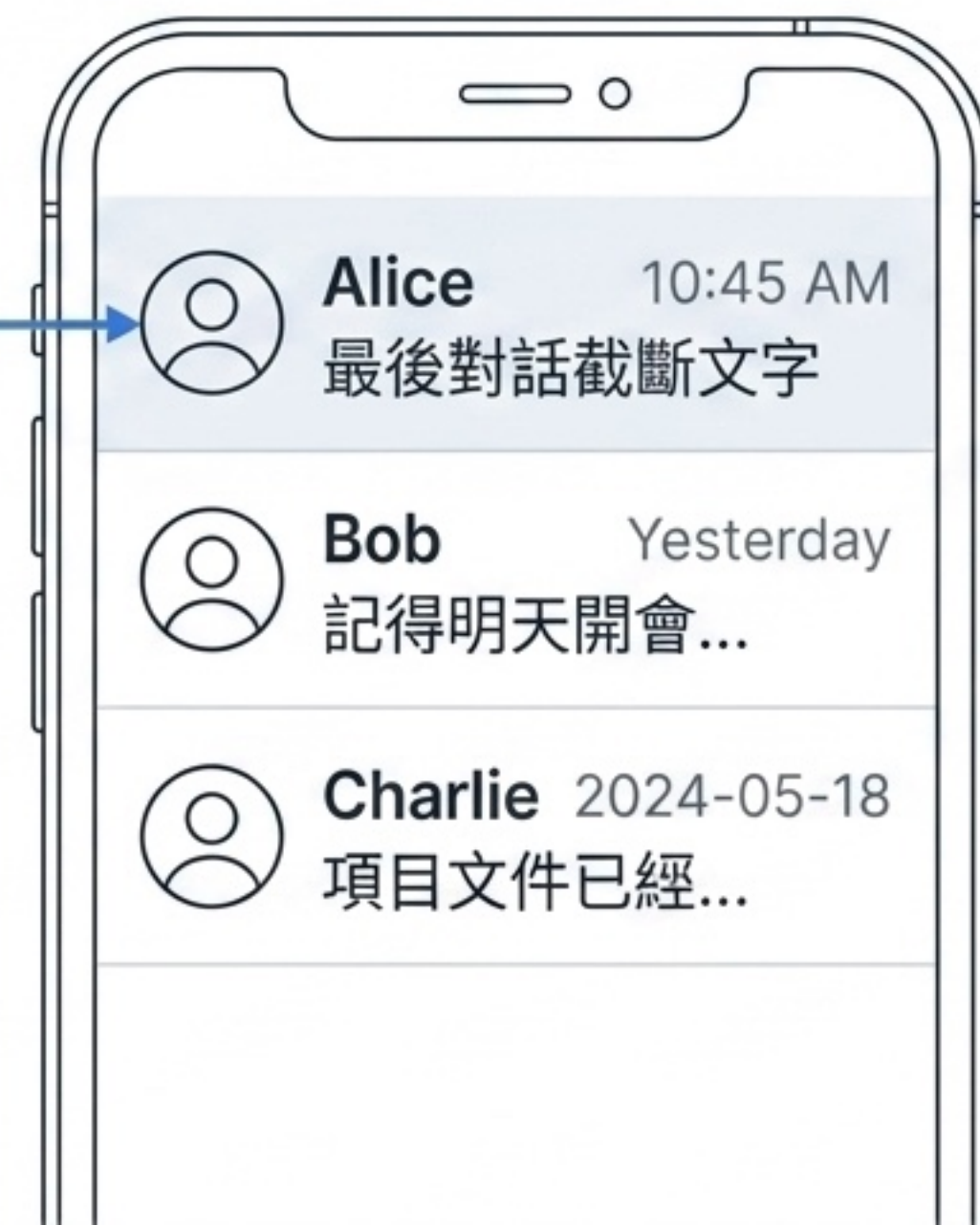
利用 key 參數與 lambda 函數，將陣列轉換成 UI 呈現的正確順序。

Code-to-UI Mapping (Backend)

```
max(chat_messages,  
    key=lambda item:  
    item["created_at"])
```

```
sorted(chats,  
    key=lambda item:  
    item["last_time"],  
    reverse=True)
```

UI Diagram (Mobile Screen)



↑
Newest
First

延伸閱讀來源

後端技術 (FastAPI & Pydantic)

- 🔗 HTTPException 錯誤處理與 CORS 跨域請求
- 🔗 Body Fields 與 Pydantic Field constraints

前端技術 (Expo & React Native)

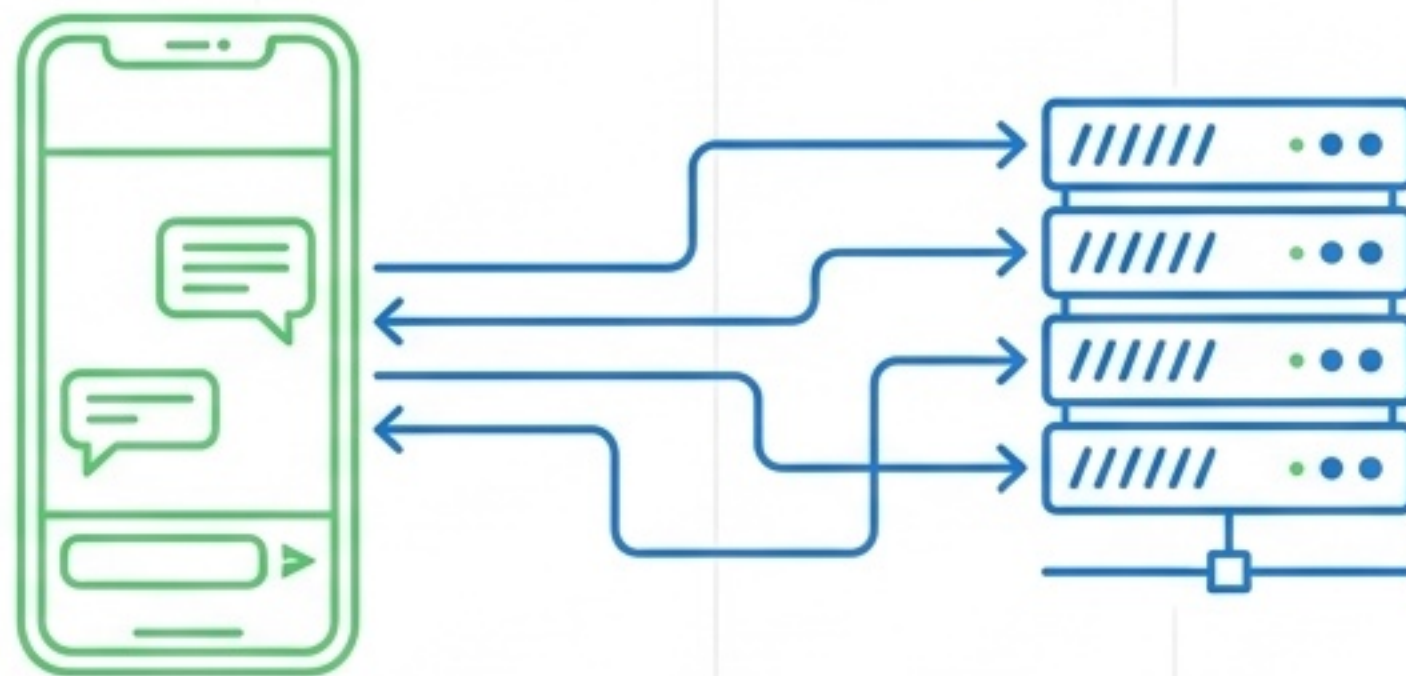
- 🔗 Expo Router (navigation, redirects) 與 Constants
- 🔗 React Hooks (useContext, useMemo, useCallback)
- 🔗 React Native Core Components (含 KeyboardAvoidingView)

其他通用知識

- 🔗 MDN Response.ok 狀態判定
- 🔗 TypeScript Utility Types 實用型別

React Native 串接 FastAPI 全端開發教學-5：進階設計

延伸補充聊天 App 需要使用的語法、函式、關鍵字、技巧與套件。



延伸主題摘要：後端架構



核心模型與驗證

- 聊天 App 領域模型 (User, Friendship, Message, ChatSummary)
- Pydantic Field 驗證 (長度限制、Optional、date)
- UUID 與 UTC ISO 時間戳



進階 API 開發

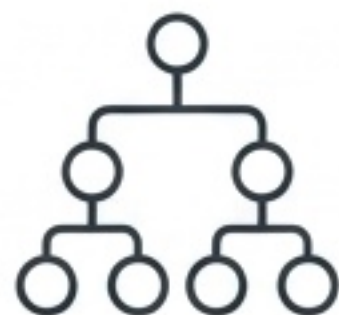
- 非 CRUD 類型 API (註冊、登入、個人資料等)
- HTTP 錯誤處理 (401, 403, 409 與 detail)
- public_user 資料遮罩防密碼外流



資料結構與測試

- 雙向好友關係 (set[tuple[str, str]])
- Python 進階語法 (list comprehension, max, sorted)
- seed endpoint 產生測試資料

延伸主題摘要：前端應用



狀態與路由

- Auth 全域狀態 (AuthContext, Provider, 自訂 hook)
- 效能優化 (useMemo, useCallback)
- Expo Router 分組 ((tabs), Redirect)



API 與資料串接

- 泛型 API Client (apiRequest<T>)
- 動態推導 API_BASE_URL (expo-constants)
- 畫面聚焦重載 (useFocusEffect)

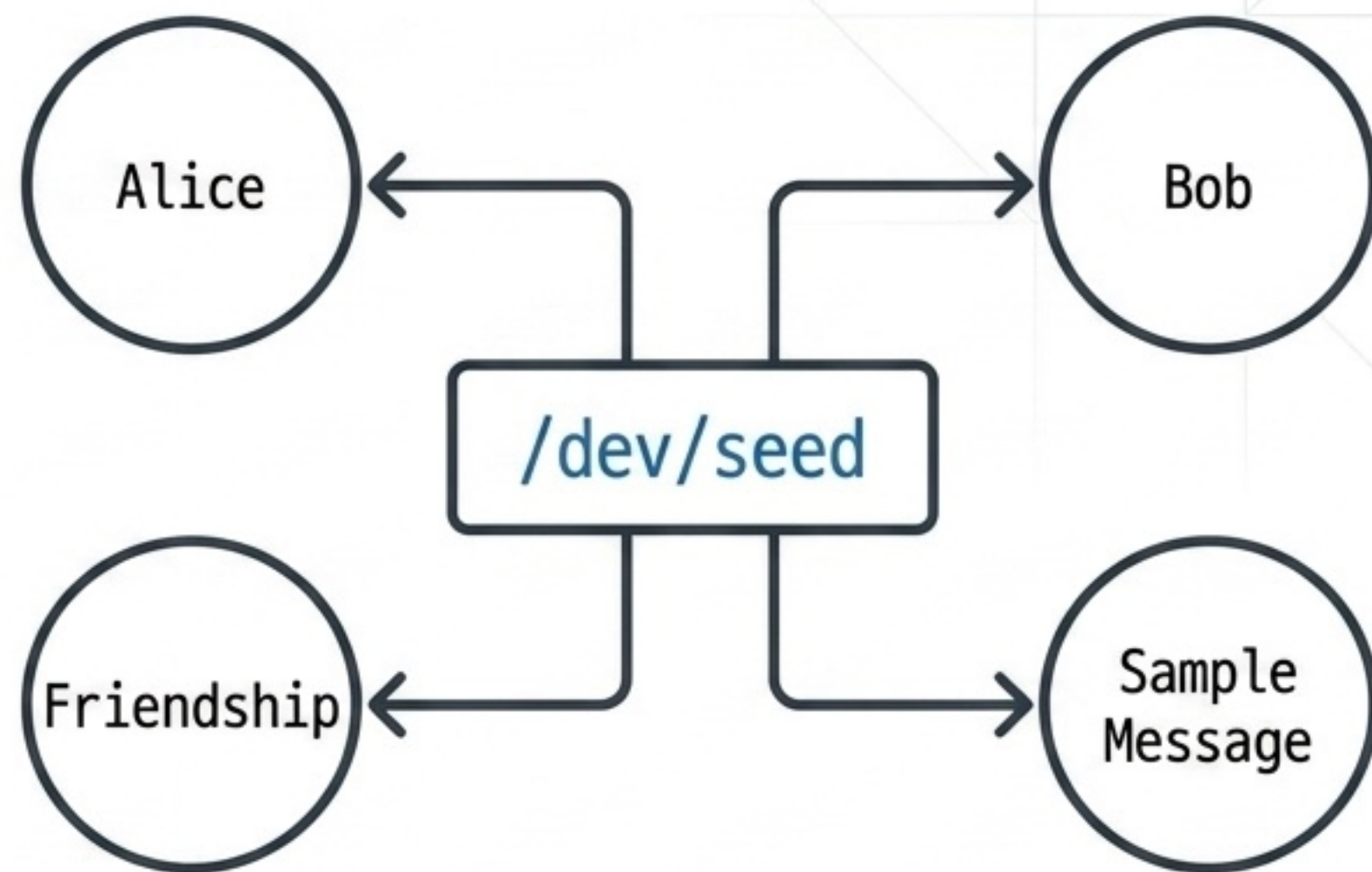


UI / UX 優化

- 安全輸入與表單屬性
- FlatList 進階控制
- 鍵盤防擋 (KeyboardAvoidingView)
- 條件渲染與樂觀更新

Slide 60 : Seed Endpoint 建立測試資料

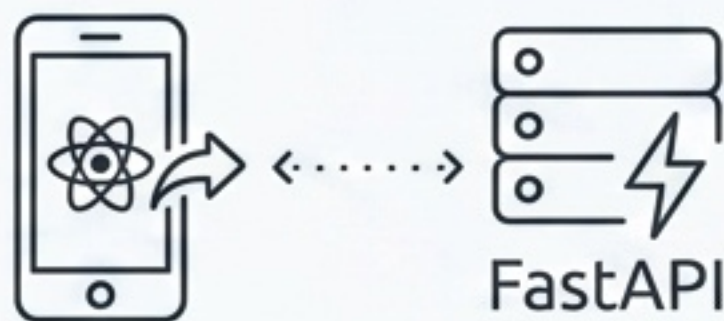
- `/dev/seed` 是課堂示範常見技巧，可快速省去重啟伺服器後手動建檔的麻煩。
- 一次點擊即可建立：Alice 使用者、Bob 使用者、雙方好友關係、一則範例訊息。



注意事項：正式產品必須避免將 dev endpoint 開放到正式環境中，只適合開發教學環境。

Slide 61：CORS 與 allow_credentials 注意事項

CORS 是前端存取不同來源 API 時的重要安全防護。allow_methods 與 allow_headers 控制允許的方法與標頭。(延伸閱讀：FastAPI CORS 官方文件)



開發環境

```
allow_origins=['*']
```

方便測試，但安全性較低



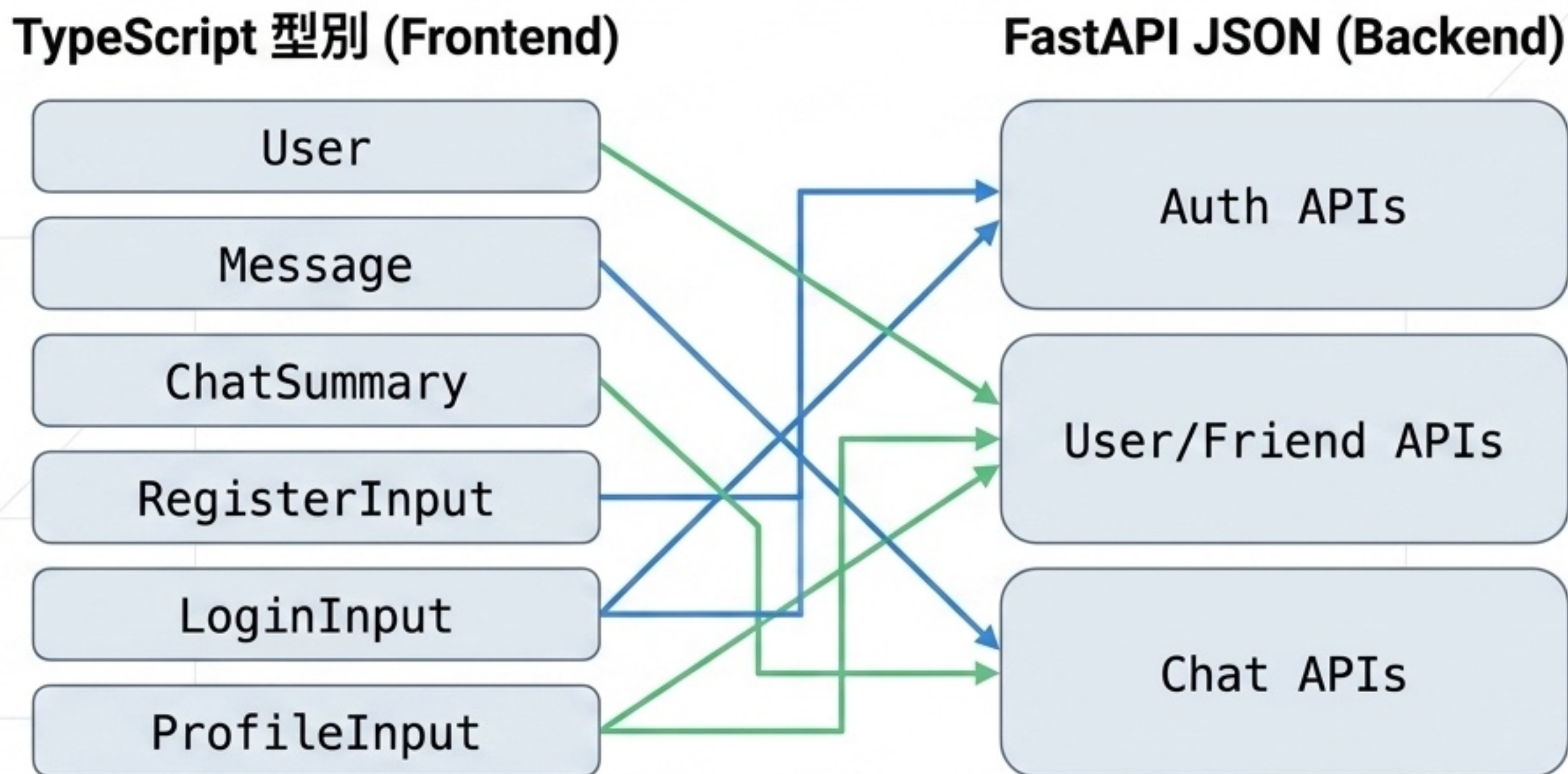
正式環境

```
allow_credentials=True
```

必須搭配明確的 origin 清單確保安全

Slide 62：前端型別升級：User Message ChatSummary

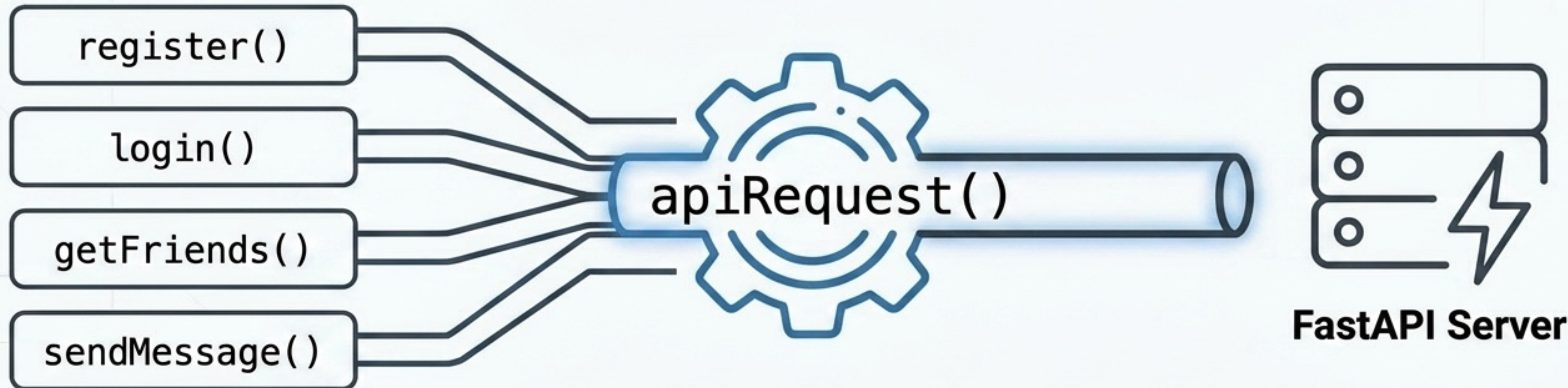
前端的 TypeScript 型別必須跟 API 回傳 (Response) 以及請求主體 (Request Body) 高度對齊。



Slide 63 : API Client : 集中管理 fetch

與其讓每個 API 各自呼叫 fetch，更完整的專案架構應導入 `apiRequest()` 集中處理重複邏輯：

- 管理 `API_BASE_URL` 與 HTTP method
- 處理 JSON headers 與 `JSON.stringify body`
- 統一判斷 `response.ok` 錯誤與解析



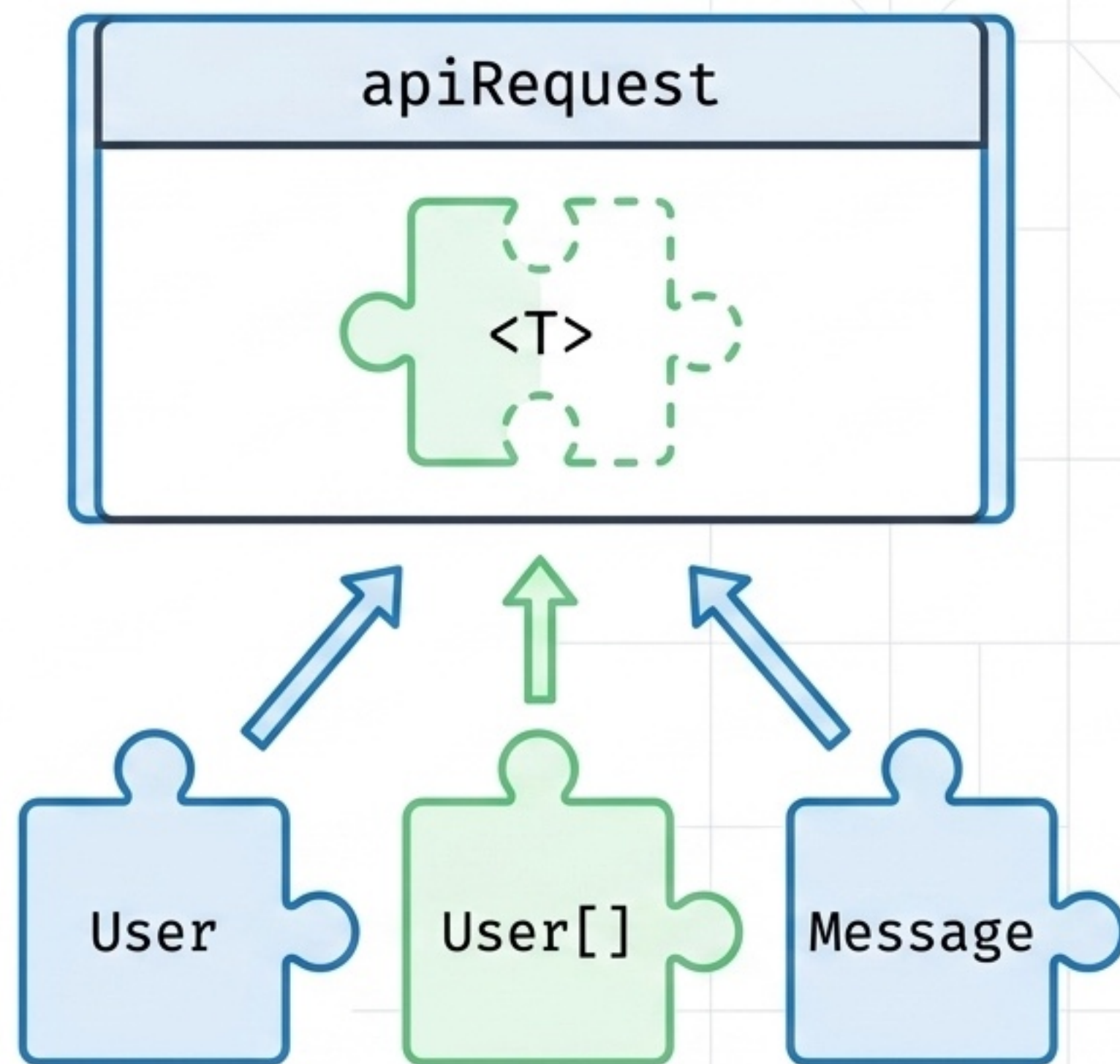
Slide 64 : TypeScript 泛型 T

在 `apiRequest<T>` 中，泛型 `T` 代表「預期會回傳的型別」。

教學價值：

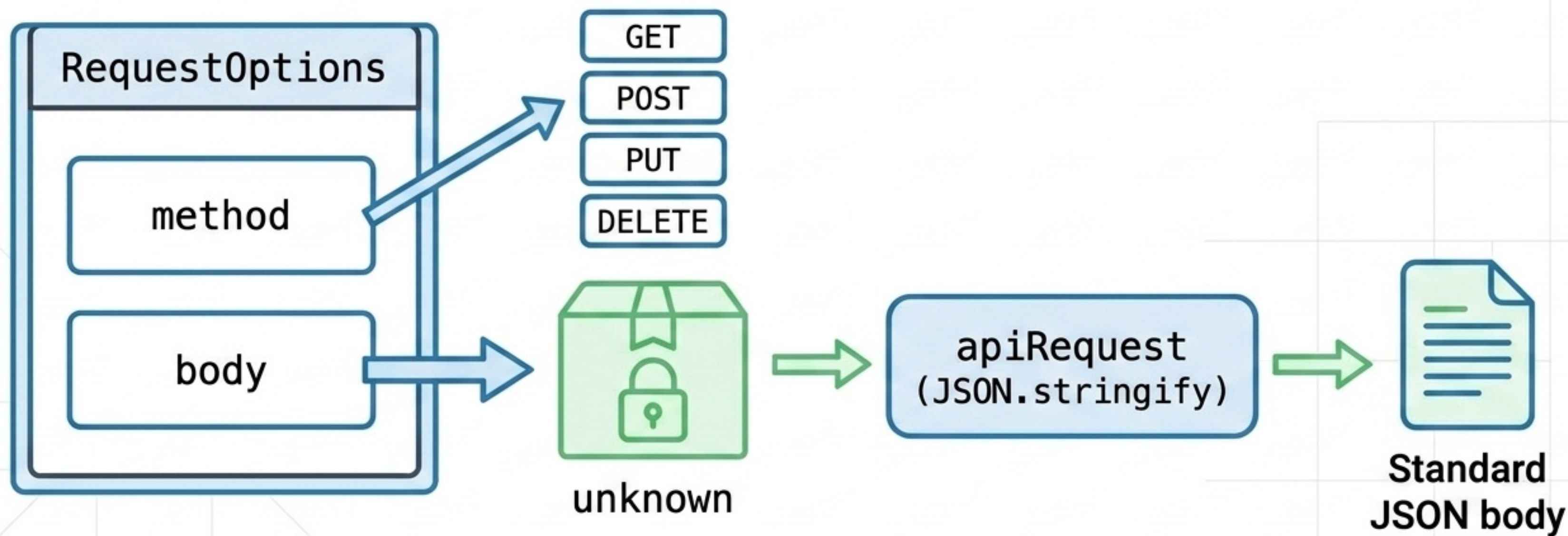
- 提供精準的型別提示 (IntelliSense)
- 讓呼叫端清楚預期資料
- 減少將 `Message` 誤用為 `User` 的錯誤

```
register()      // 預期 User  
getFriends()   // 預期 User[]  
sendMessage()  // 預期 Message
```



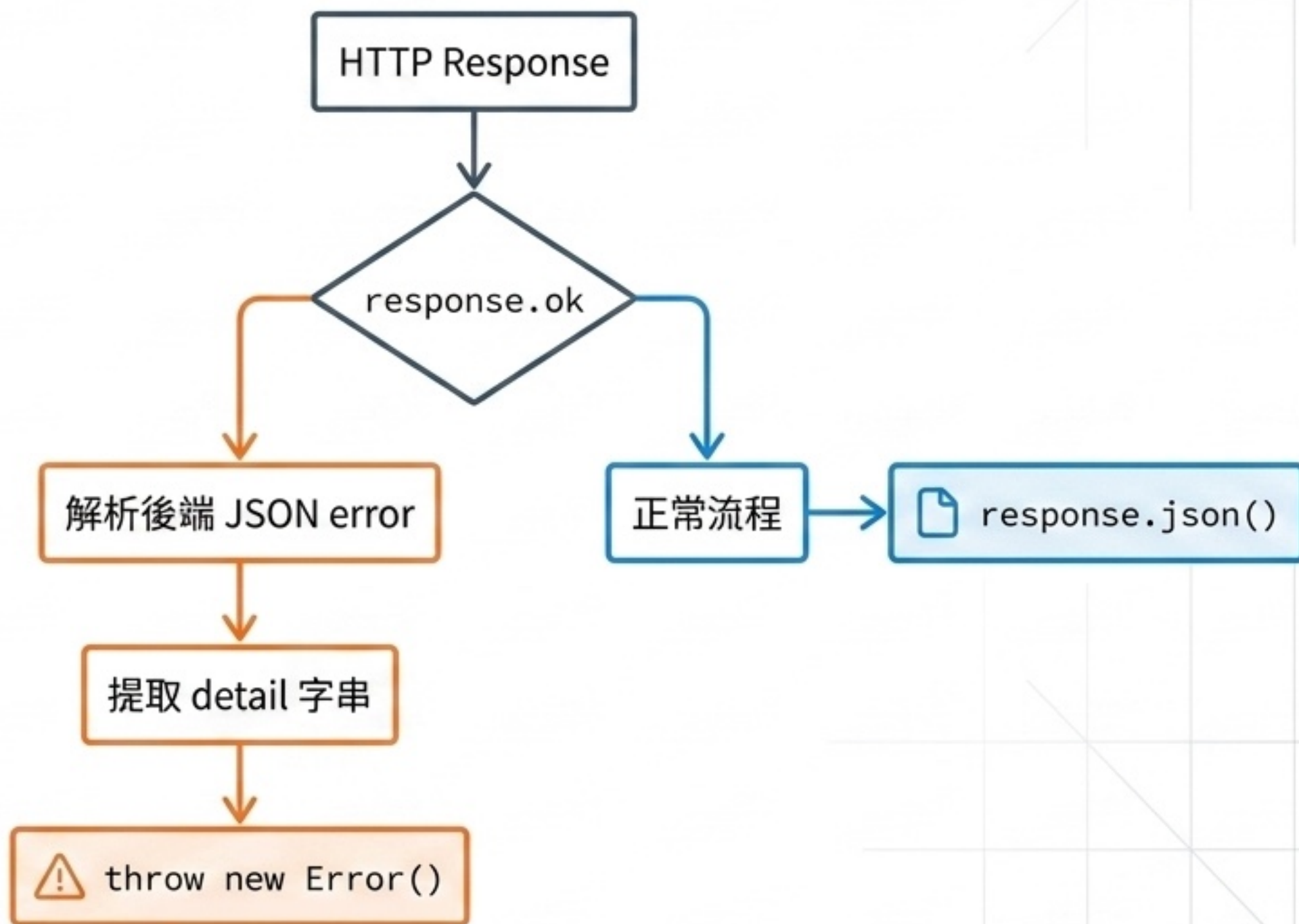
Slide 65 : RequestOptions 與 unknown

RequestOptions 是 fetch 設定物件的簡化版。傳入的 body 設為 unknown，這比 any 更適合教學，因為 unknown 能強制保留開發時的安全性，而不會關閉型別檢查。



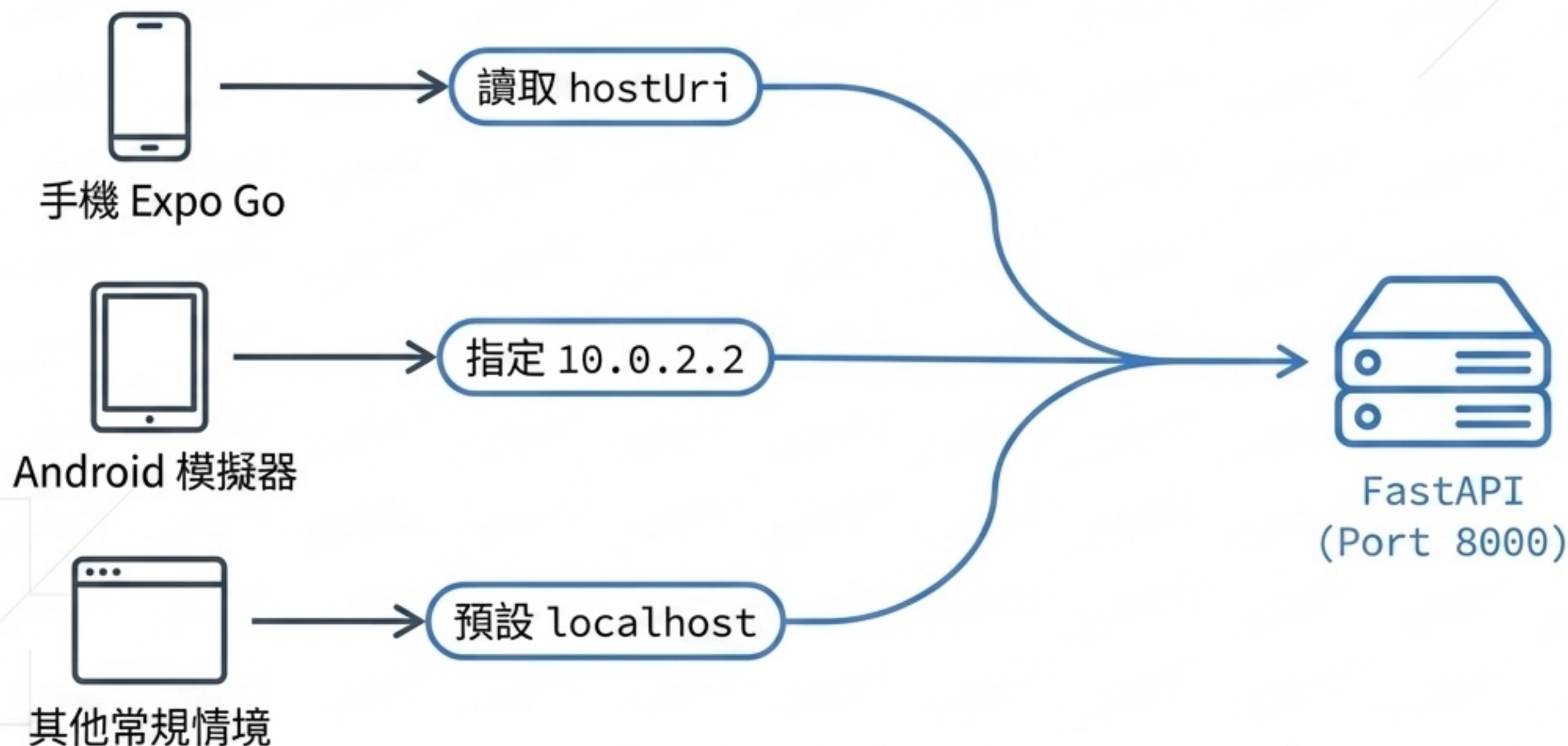
Slide 66 : response.ok 與錯誤訊息

fetch 不會因為 HTTP 404/409 錯誤狀態碼自動進入 catch，前端必須主動檢查 response.ok。
(延伸閱讀：[MDN Response.ok](#))



Slide 67：自動推導 API_BASE_URL

為解決「手機連不到開發環境」的痛點，專案利用 expo-constants 與 Platform 動態推導連線位址，取代手動設定 IP。(延伸閱讀：[Expo Constants 官方文件](#))



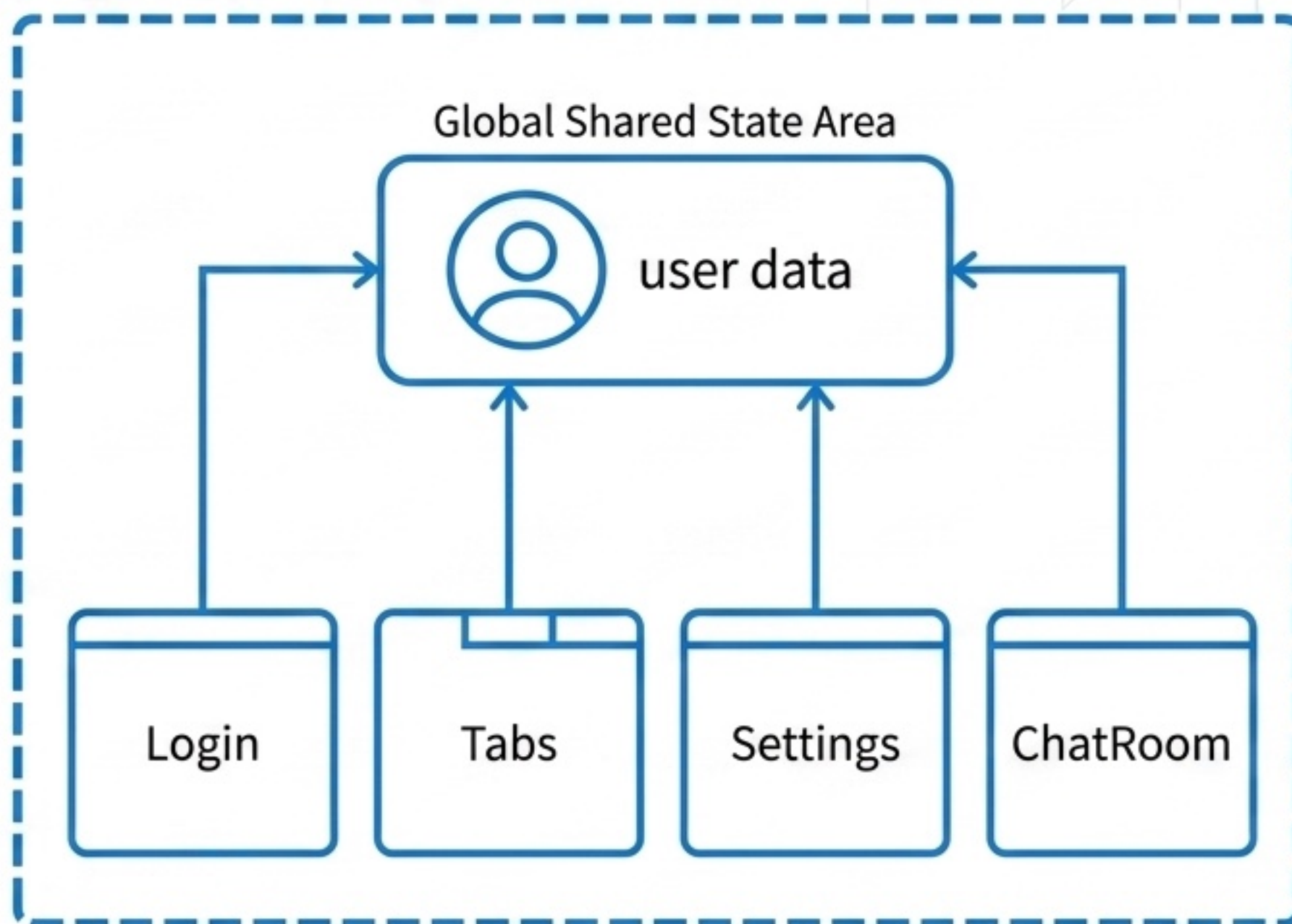
Slide 68 : AuthContext : 全域登入狀態

登入後，許多頁面都需要讀取「當前使用者 (user)」。若每一頁都依賴 props 傳遞，程式將難以維護。

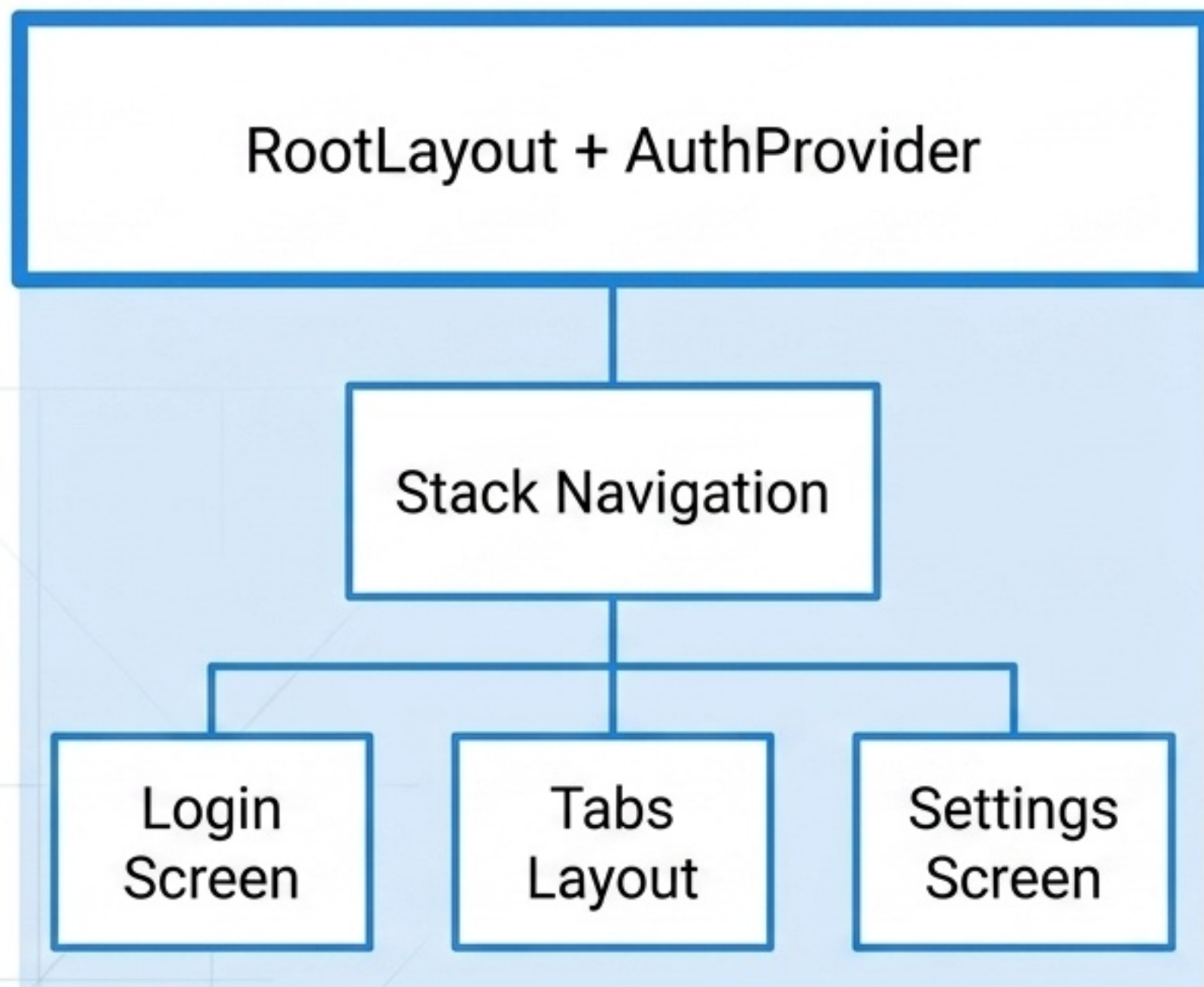
AuthContext 的核心用途：

- 儲存目前 user
- 提供 `signIn()` 與 `signOut()`
- 提供 `setUser()` 更新個人資料

AuthProvider



Slide 69 : createContext 與 Provider



createContext 建立隱形的全域資料通道，Provider 負責將數值廣播給子元件。

在 RootLayout 最外層包覆 AuthProvider：

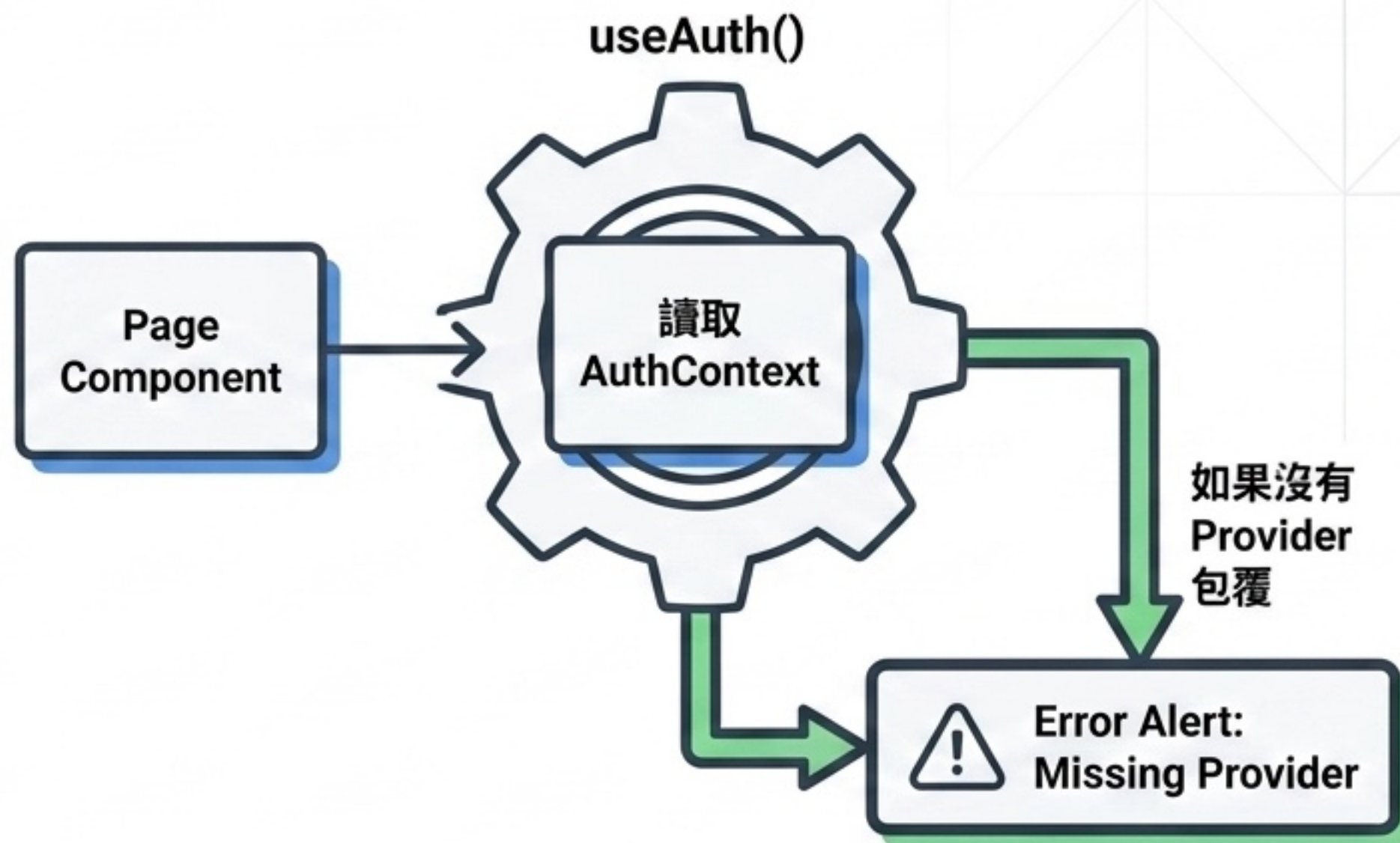
- 登入頁可以 `signIn`
- `Tabs` 可以判斷是否登入
- 設定頁可以 `setUser`

Slide 70：自訂 useAuth Hook

useAuth() 自訂 Hook 封裝了
useContext(AuthContext)，讓頁面
無需直接操作底層。

內建防護機制：

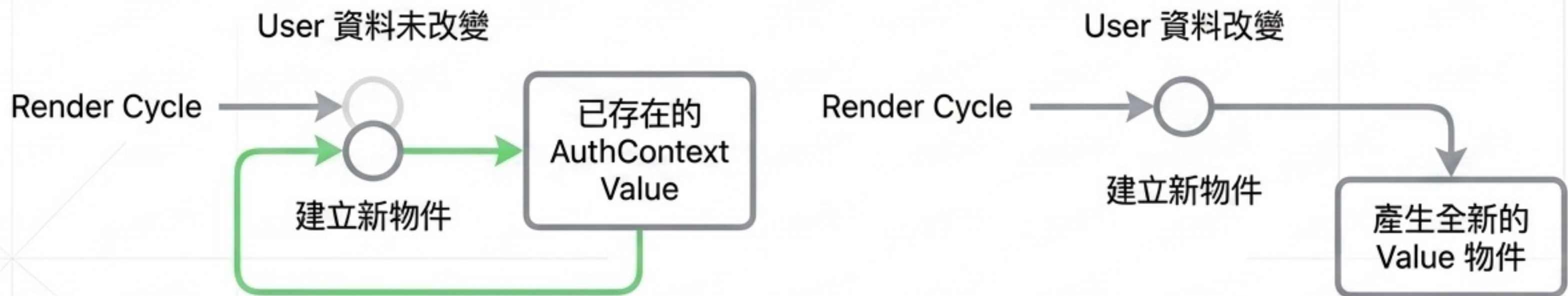
- 若元件不小心被放在 AuthProvider 範圍之外
- Hook 會立即 throw Error
- 幫助開發者快速發現並修復架構錯誤



Slide 71 : useMemo 穩定 Context Value

在 AuthProvider 中使用 useMemo，目的並非「快取複雜計算」，而是為了維持 Context Value 物件參考的穩定性。

React 預設每次 render 都建立新物件。useMemo 在依賴項不變時重用舊物件，避免子元件無意義的重新渲染。

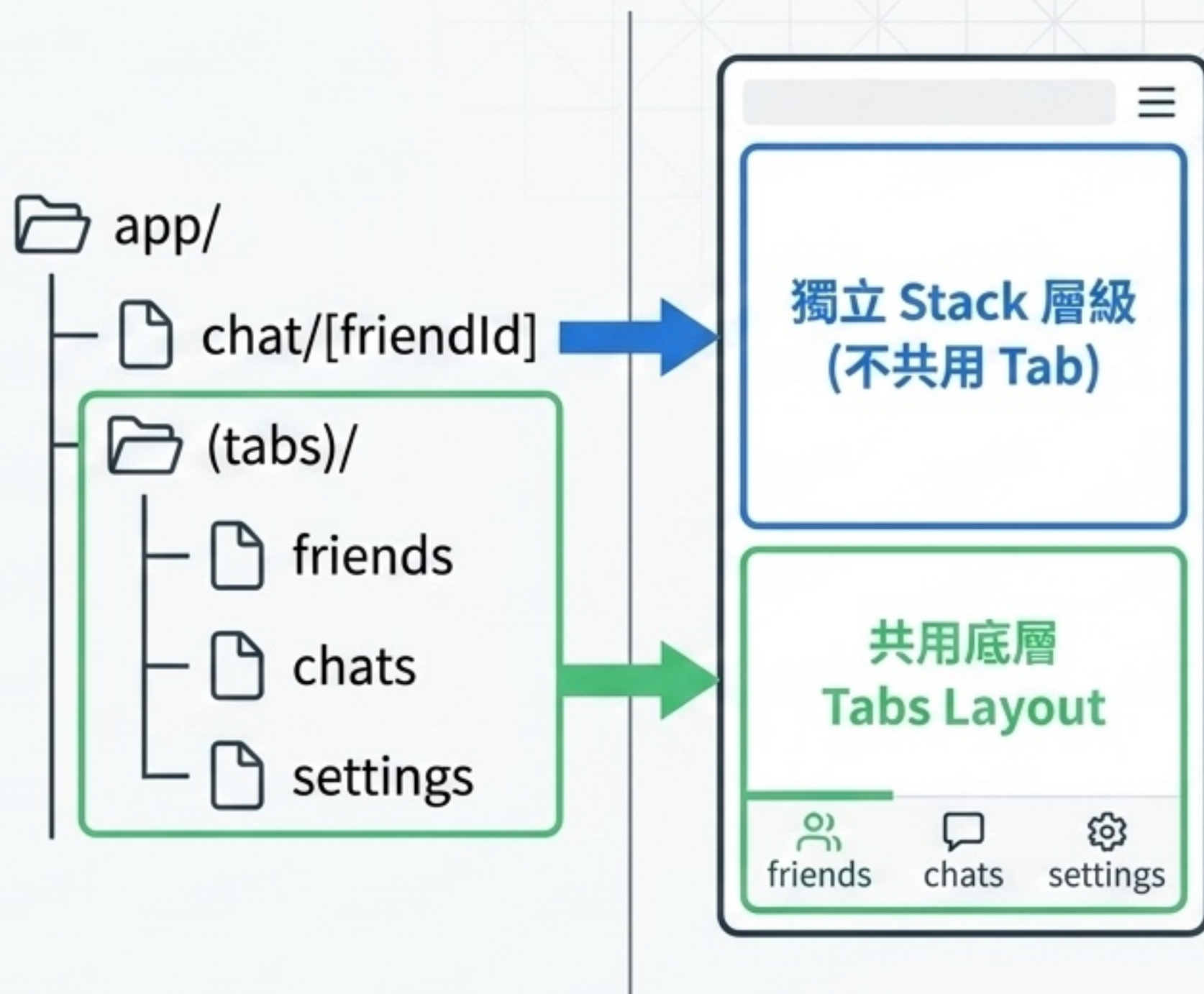


Slide 72 : Route Group : (tabs)

在 Expo Router，括號命名的
app/(tabs)/ 是 Route Group。

特性：資料夾名稱不會出現在網址
中，純粹用於組織版面。

- friends、chats、settings
共用 Tabs
- chat/[friendId] 抽離至
Stack 顯示全螢幕詳細頁



Slide 73 : Tabs 導航

聊天 App 核心採用 Tabs（標籤式）建立分頁：好友列表、聊天列表、個人設定。

設計抉擇：

- Tabs 適合長時間停留、頻繁切換的區域。
- Stack（堆疊式）適合登入、註冊或「聊天室對話」這類有明確進出 (Push/Pop) 的頁面。



Slide 74 : Ionicons 圖示套件

導入 @expo/vector-icons 的 Ionicons 以提升導航專業感。

- tabBarIcon 本質是回傳元件的函式。Expo Router 底層會自動傳入當前的 color 與 size。
- 開發者不需寫 if/else 判斷，圖示即可依據選取狀態 (active) 自動套用 active tint color。





React Native 串接 FastAPI 全端開發教學-6：測試除錯

延伸補充聊天 App 需要使用的語法、函式、關鍵字、技巧與套件。

延伸主題摘要：後端



資料與驗證模型

領域模型：`User`、`Friendship`、`Message`、`ChatSummary`

Pydantic Field 驗證：`min_length`、`max_length`、`Optional`、`date`

資料庫工具：`UUID` 產生 ID、UTC ISO 時間戳



API 設計實務

非 `CRUD` 端點：註冊、登入、個人資料、好友、聊天訊息

`Seed Endpoint`：產生課堂測試資料



安全性與狀態碼

HTTP 錯誤：`401`、`403`、`409` 與 `detail` 錯誤訊息

資料遮罩：`public_user` 回傳避免洩漏 `password`



Python 進階語法

集合：`set[tuple[str, str]]` 表示雙向好友關係

操作：`list comprehension`、`max(key=...)`、`sorted(key=..., reverse=True)`

延伸主題摘要：前端



狀態與路由管理

狀態控制：AuthContext、Provider、useContext、useMemo、客製 useAuth

Expo Router：(tabs)、Tabs、Redirect、Stack.Screen



畫面生命週期與 API

Hooks：useFocusEffect 與 useCallback (聚焦時重新載入)

泛型設計：apiRequest<T>、RequestOptions、Promise<T>

環境變數：hostUri 與 Platform.OS 動態推導

API_BASE_URL



表單與列表進階 UI

安全輸入：secureTextEntry、autoCapitalize、multiline

FlatList：ListEmptyComponent、numberOfLines、contentContainerStyle



使用者體驗 (UX)

佈局適應：KeyboardAvoidingView、SafeAreaView

視覺防呆：Image 頭像與 fallback UI

互動回饋：條件渲染、樂觀更新、錯誤恢復、loading disabled

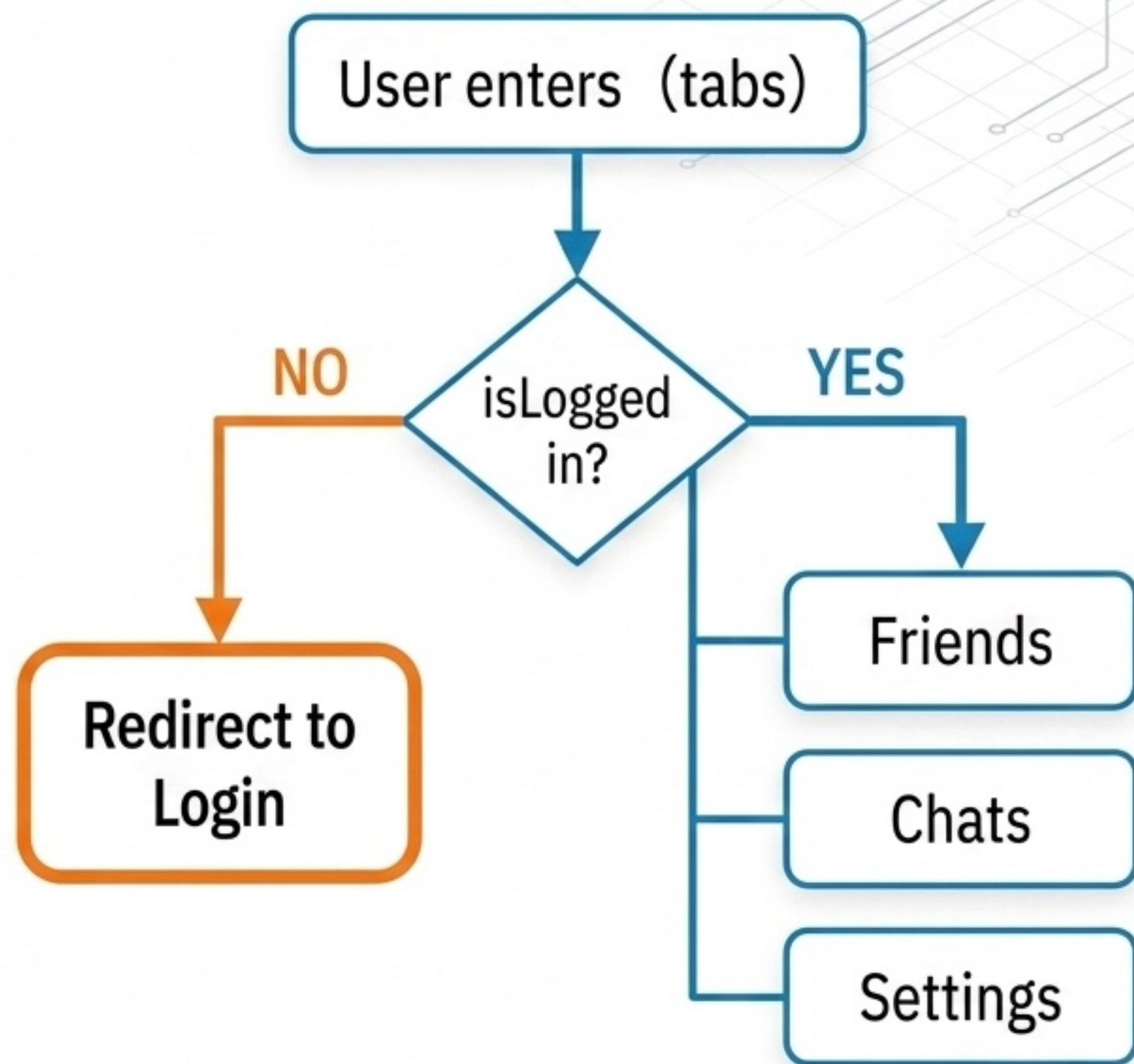
Redirect：保護登入後頁面

TabLayout 會先檢查 user 狀態。如果沒有登入，就回到首頁。

前端路由保護情境：

- 未登入不能看到好友列表
- 未登入不能看到聊天列表
- 未登入不能進設定頁

⚠ 注意事項：前端 Redirect 旨在提升使用者體驗，但後端仍必須做資料權限檢查。



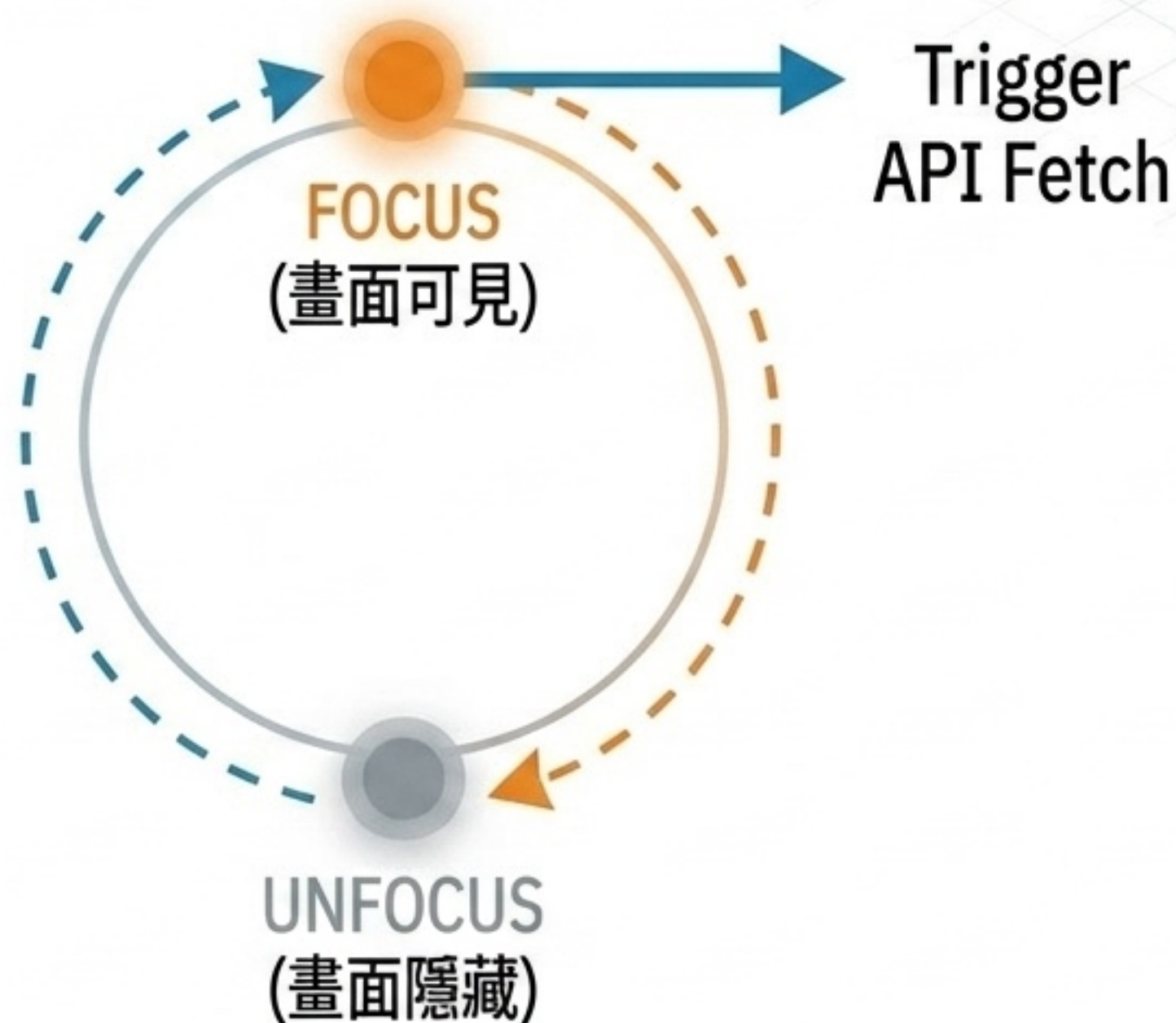
useFocusEffect：畫面回來時重新載入

有別於以往使用 `useEffect`，聊天 App 建議使用 `useFocusEffect`，確保畫面每次重新聚焦時都能獲取最新狀態。

最佳適用情境：

- 👉 從聊天室返回聊天列表 → 更新最後一則訊息摘要
- 👉 新增好友後返回好友列表 → 即時顯示最新好友
- 👉 在不同 Tab 間切換回來時 → 重新 fetch 資料
- 👉 在不同 Tab 間切換回來時 → 重新 fetch 資料

Expo 官方建議：Callback 應搭配 `useCallback` 避免過度執行。

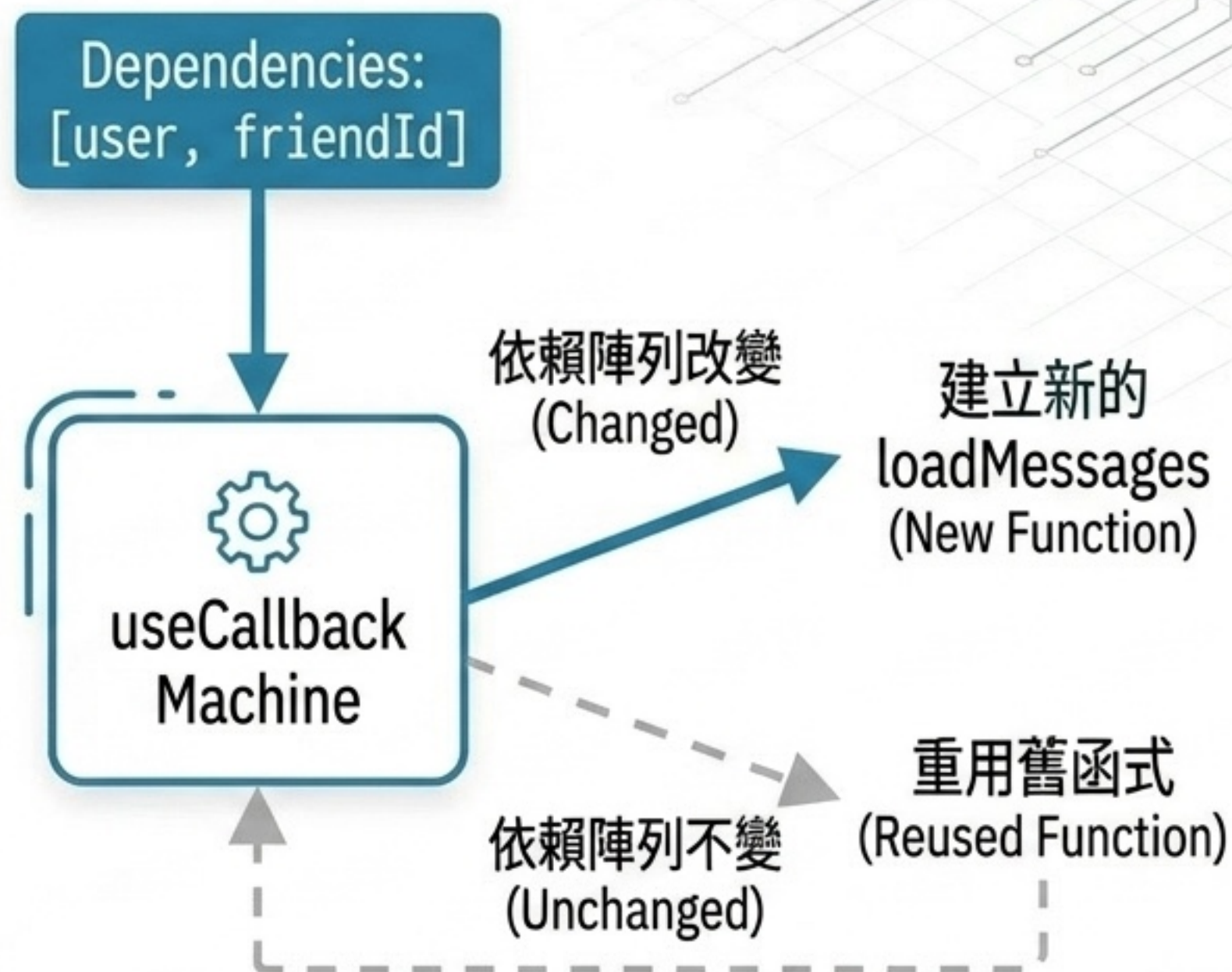


useCallback：穩定函式依賴

`useFocusEffect` 內的 `callback` 需要保持穩定，可將 `loadFriends` 與 `loadMessages` 包裝成 `useCallback`。

當 Dependencies 沒變時 → 重用同一個函式
當 Dependencies 改變時 → 才會建立新函式

常用情境：effect、focus effect、傳遞給子元件的 handler



Link asChild 與 Pressable

在 Login 頁面中，可使用 `Link`
`href='/register' asChild`
將 `Pressable` 包覆。

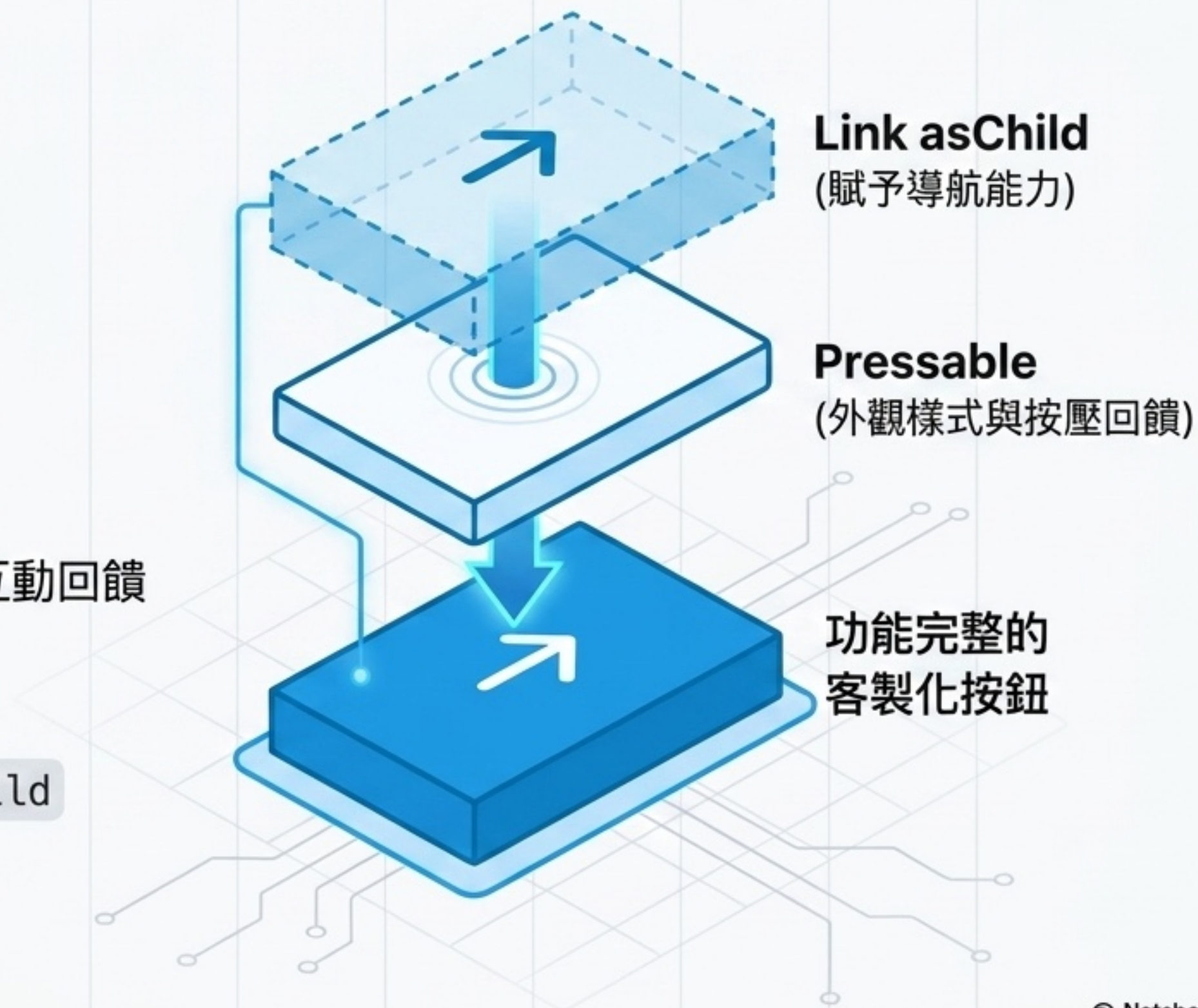
雙重優勢：

在選到 Pract Router 的包覆：

- 獲取 Expo Router 的連結導航能力
- 保留 React Native Pressable 的按壓樣式與互動回饋

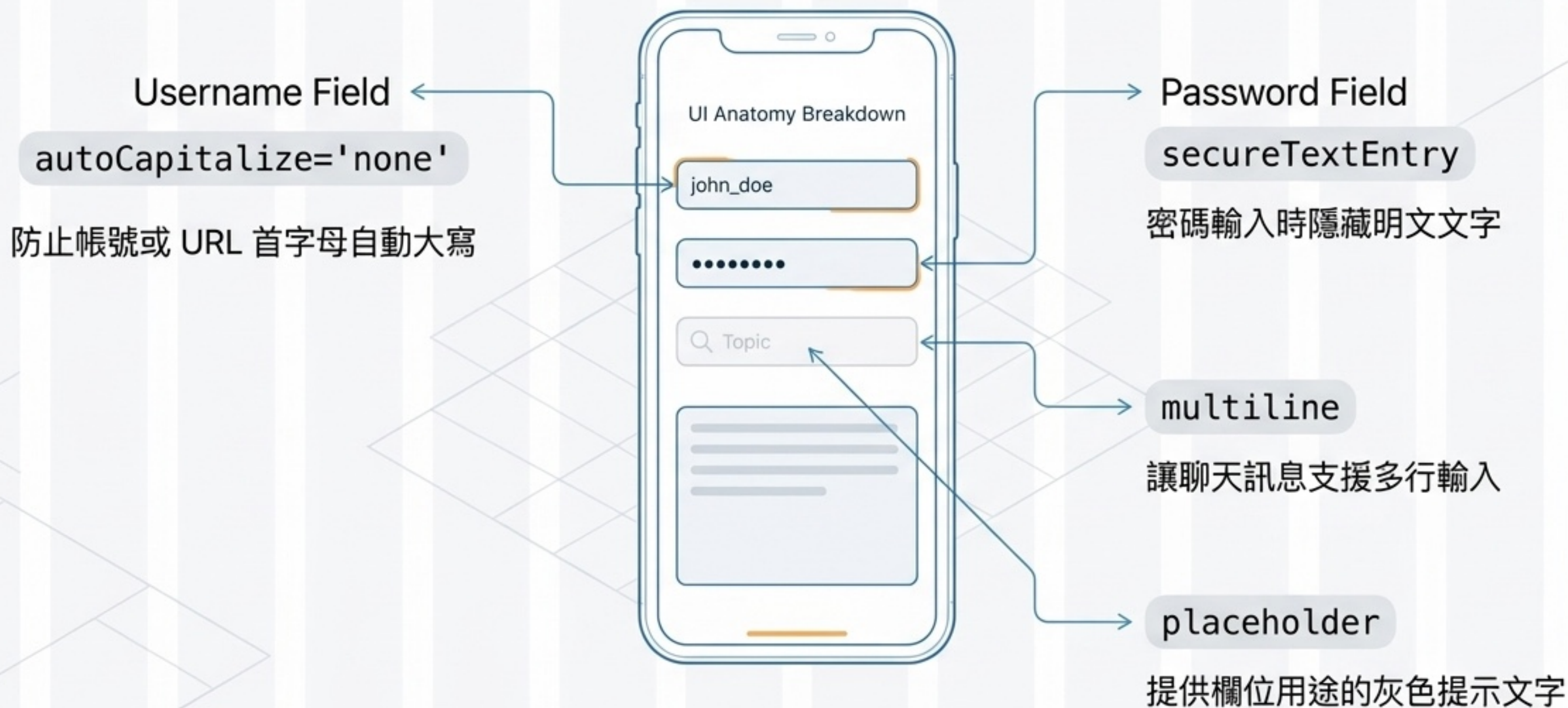
進階控制：

`Link` 預設將內容放置於 `Text` 內，使用 `asChild`
屬性可將行為完整傳遞給自訂子元件。



表單輸入的實務屬性

聊天 App 中的 TextInput 可透過實務屬性大幅提升使用者體驗（非關資料邏輯，但影響深遠）：



FlatList 進階顯示技巧

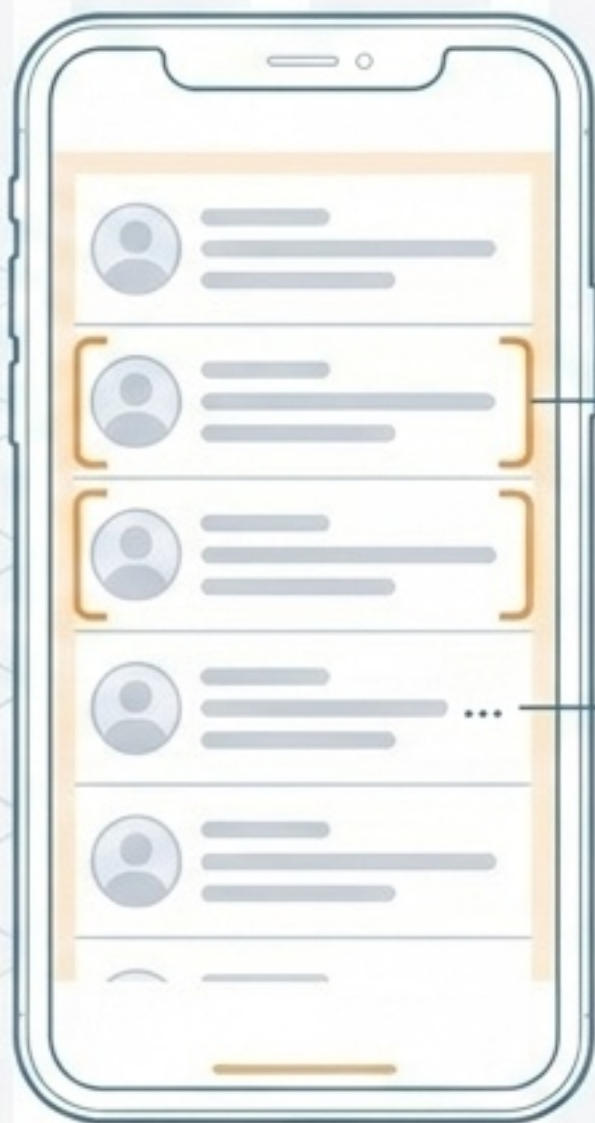
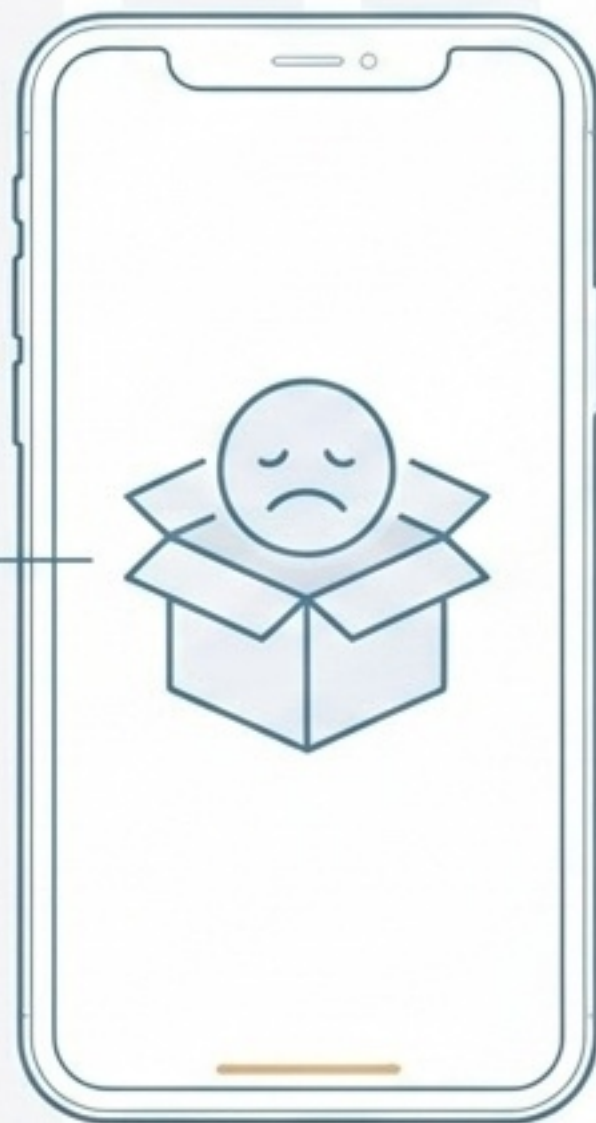
聊天 App 的列表需求較為複雜，更需要考量空資料與極端長度的排版：

無資料狀態 (Empty State)

有資料狀態 (Populated List)

ListEmptyComponent

當沒有好友或聊天紀錄時，
顯示友善的空狀態 UI



contentContainerStyle

精確設定列表的內距
(Padding) 與元件間隔 (Gap)

numberOfLines={1}

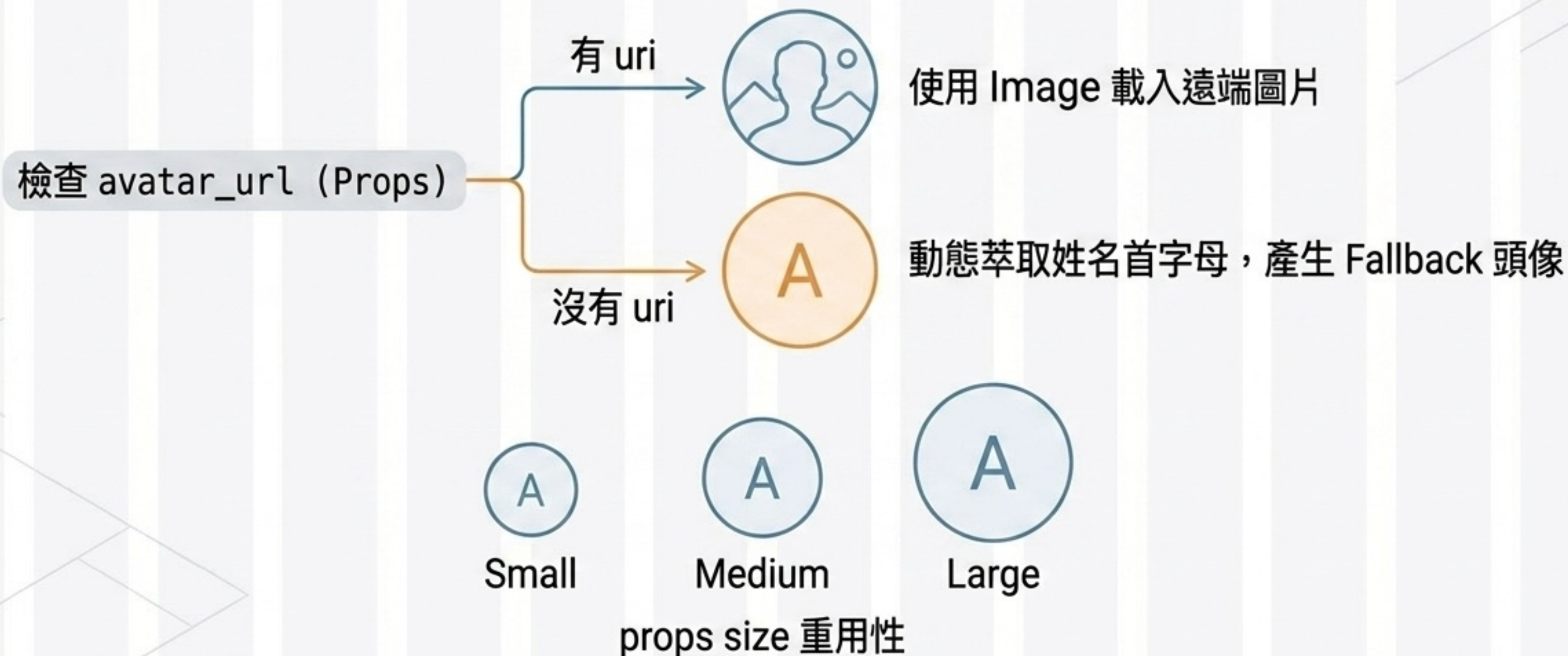
限制聊天摘要僅顯示一行，
過長自動截斷

keyExtractor：確保使用穩定的 ID 作為列表渲染的 Key

UserAvatar：圖片與 fallback UI

UserAvatar 元件的設計展現了元件化思維與防呆機制。

封裝重複 UI 成為高重用性元件，透過 props 彈性控制 name、uri、size，讓全端多個頁面無縫共用。

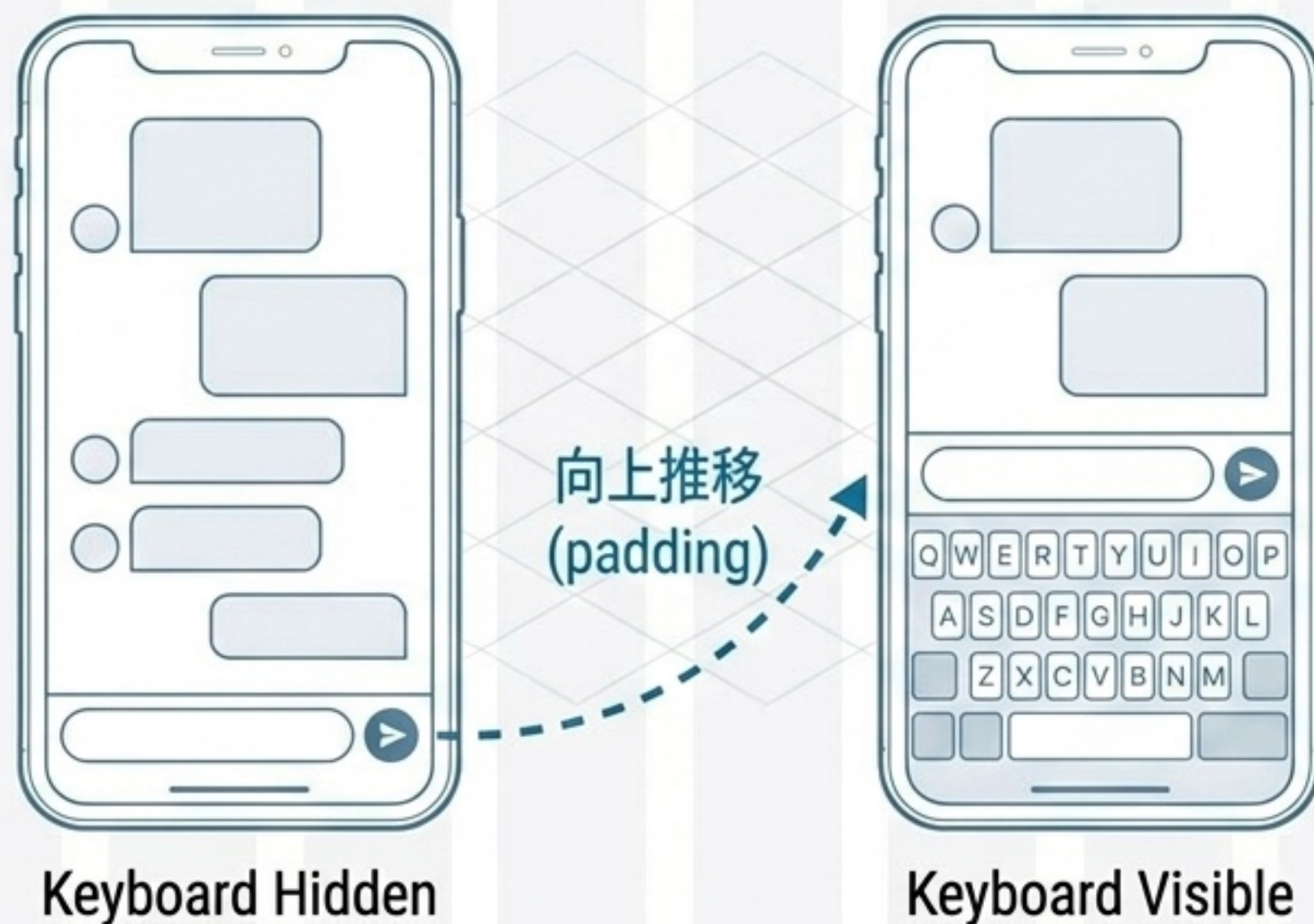


KeyboardAvoidingView：聊天輸入列

痛點：手機鍵盤彈出時，位於畫面底部的輸入框容易被遮擋。

解決方案：聊天室畫面包覆 `KeyboardAvoidingView`，確保 `inputBar` 固定於底部。

跨平台策略：iOS 設定 `behavior='padding'`；Android 設為 `undefined` 避免版面破圖。



送出訊息：樂觀更新與錯誤恢復

比單純重新 fetch 更有互動感，將使用者體驗與錯誤處理整合設計。



延伸課程實作任務

請以 CRUD 基礎，動手完成聊天 App 延伸功能：

建立資料模型與 API (註冊/登入)

完成好友與聊天功能 (列表、收發)

實作架構 (AuthContext、Tabs、useFocusEffect)

完善 UX (錯誤訊息與空狀態)

資料模型與
API

AuthContext
狀態管理

App 畫面與
UX 實作

未來展望
(Next Stage)

下一階段：整合
SQLite、真實資
料庫連線、
密碼雜湊 (Hash)
與 JWT 驗證